Relational Database Design

A Concept-First Book (No SQL Required)

Diogo Ribeiro

February 2, 2026

Preface

Relational database design is a discipline of meaning. It is about deciding what exists, how it relates, and what rules must always hold. This book treats design as a conceptual activity before any SQL dialect or storage technology. It assumes you want models that are stable, explainable, and correct.

The approach here is deliberately sober. We use short sentences when clarity matters and longer ones when the idea needs context. We use examples to anchor meaning, and constraints to make that meaning enforceable. The goal is not to draw the prettiest diagrams. The goal is to build models that survive reality.

This book is not about performance tuning, indexing, or query optimization. Those are important, but they come later. If the conceptual model is wrong, no amount of optimization will save it. If the model is right, implementation becomes a disciplined translation.

The material grew from real projects in healthcare, education, and product systems. The recurring lesson is that ambiguity is the real enemy. A model that reads like a clear set of rules is already most of the way to a reliable system.

I hope this book helps you build that kind of model.

How to Use This Book

This book is designed to be read in order, but it also supports targeted use. If you are new to modelling, start from the beginning and follow the workflow. If you already model data, jump to the chapters on constraints, validation, and review.

Each chapter follows a simple pattern. You will see definitions, examples, and short exercises. The examples are deliberately small so the rules are visible. Use them as templates, not as complete systems.

At the end of each chapter, pause and write a few rules in your own domain. Even a short exercise will reveal gaps in meaning. When the rules are clear, the schema is usually straightforward.

If you are working in a team, use the diagrams and rule lists as shared artifacts. A good conceptual model is a communication tool as much as it is a design.

Contents

Chapter 1

The Design Workflow

Learning objectives

In this chapter you will learn a repeatable way to design relational databases without relying on SQL. You will be able to move from a domain description to a schema that is justified by rules and validated by questions.

1.1 What database design is

Relational database design is often introduced as "creating tables". That framing is convenient, but it is not accurate. Tables are a *representation*. Design is the work that happens before representation becomes safe.

When design is weak, the system still runs, but meaning becomes unstable. Facts get duplicated, updates require manual discipline, and invalid states slip into the data because nothing prevents them. In practice, most "data problems" in organisations are design problems: the model did not make the rules explicit.

Definition

A *relational design* is a model of a domain that preserves meaning and supports correctness. It does this by making identity explicit and by enforcing (or supporting enforcement of) the domain rules as constraints.

The key idea is simple: data is not just stored, it is *constrained*. A schema is a statement about what is allowed to exist. If the schema allows the wrong states, the application will eventually contain wrong states.

1.2 The workflow as a discipline

A beginner often asks: "Where do I start?" The correct answer is: you start with the domain, not with the database. You describe the world you are modelling, then you force that description to become precise. Only then do you translate it into relational structures.

Principle

Use this workflow throughout the book:

$$\text{Story} \rightarrow \text{Business rules} \rightarrow \text{Conceptual model} \rightarrow \text{Relational schema} \rightarrow \text{Validation}$$

This workflow is not a bureaucratic ritual. It exists because each step removes a different kind of ambiguity. A story is expressive but vague; rules are precise but incomplete; a conceptual model gives structure; the relational schema commits to enforceable choices; validation tests whether you actually captured the domain.

1.3 Step 1: Write a small, realistic story

A story is not a requirements document. It should be short, concrete, and close to what people say in real life. The goal is to capture the domain in a way that a reader can challenge.

Example

Clinic story (minimal)
A clinic has doctors and patients. Patients book appointments with doctors. Each appointment has a start time. A doctor cannot have two appointments at the same time.

Stories can be short in other domains too.

Example

University story (minimal)
A university offers courses. Students enroll in courses each semester. An enrollment records the date and final grade. A student cannot enroll in the same course twice in the same semester.

Notice what is *not* in the story: edge cases, exceptional policies, performance concerns, and implementation details. Those belong later. If you start with them, you will build complexity before you have correctness.

1.4 Step 2: Convert the story into business rules

Stories are valuable because they contain meaning, but they are dangerous because they allow interpretation. The next step is to write rules that are testable. A rule must be something that can be true or false for a given database state. If a statement cannot be tested, it will not be enforceable, and it will not guide design decisions.

Definition

A *business rule* is a testable statement that must always hold for the system to be correct. A good rule is unambiguous, stable, and written in plain language.

From the clinic story we can extract a small set of rules. These are not "final"; they are the first precise version of the domain.

Example

Business rules (clinic)
R1: A patient can have many appointments.
R2: A doctor can have many appointments.
R3: An appointment is with exactly one patient.
R4: An appointment is with exactly one doctor.
R5: Every appointment has a start time.
R6: A doctor cannot be double-booked at the same start time.

Two points matter here. First, the words "many" and "exactly one" are doing real work: they imply cardinality and optionality. Second, rule R6 is not merely descriptive; it is a constraint that should prevent invalid states.

Deliverables

For each domain chapter, aim for 10–20 rules. At minimum, include: one uniqueness rule, one optionality rule, and one time-related rule. If your rules do not include these, your model will usually miss critical constraints.

Principle

Write rules before drawing structures. If a rule is not written, it is not enforceable.

1.5 Step 3: Build a conceptual model

Rules are precise, but they are not structured. A conceptual model gives structure by identifying what exists and how it connects.

At this stage you decide what the entities are, what attributes belong to them, and which relationships exist between entities. You also decide cardinalities: whether a relationship is one-to-one, one-to-many, or many-to-many. These decisions are not about database syntax. They are about meaning.

For the clinic, the conceptual picture is straightforward. The domain contains doctors, patients, and appointments. Appointments connect one doctor to one patient at a specific time. The relationships "patient has appointments" and "doctor has appointments" are both one-to-many. That conceptual understanding is the foundation for the relational schema.

1.6 Step 4: Translate into a relational schema (still no SQL)

The relational schema is where you commit. Entities become relations, identity becomes keys, and relationships become references. If your conceptual model is clear, this translation is not creative; it is disciplined.

This book uses a compact notation:

$$\begin{aligned}&\textbf{Appointment}(\underline{\textit{appointment\_id}}, \textit{start\_time},\\ &\qquad \textit{patient\_id} \to \textbf{Patient}, \textit{doctor\_id} \to \textbf{Doctor})\end{aligned}$$

The underline indicates the primary key, and the arrow indicates a reference to another relation.

In the clinic, the schema is small:

$$\textbf{Patient}(\underline{\textit{patient\_id}}, \textit{name}, \textit{birth\_date})$$

$$\textbf{Doctor}(\underline{\textit{doctor\_id}}, \textit{name}, \textit{specialty})$$

$$\begin{aligned}&\textbf{Appointment}(\underline{\textit{appointment\_id}}, \textit{start\_time},\\ &\qquad \textit{patient\_id} \to \textbf{Patient}, \textit{doctor\_id} \to \textbf{Doctor})\end{aligned}$$

The important work is not the existence of these relations. It is the constraints implied by the rules. For example, rules R3 and R4 imply that the references from **Appointment** to **Patient** and **Doctor** are mandatory. Rule R6 implies a constraint that prevents two appointments from using the same doctor at the same start time. In a real database, the enforcement mechanism depends on how time is represented (discrete slots or intervals), but the modelling intention is already clear.

Note

Do not add SQL early to "make it feel real." SQL can hide weak modelling decisions behind syntax. If the meaning is unclear now, SQL will not fix it later.

1.7 Step 5: Validate the design without SQL

A schema is not correct because it "looks right". It is correct because it supports the rules and the questions the domain must answer. Validation is how you prevent elegant-looking diagrams from becoming wrong systems.

One form of validation is question-driven. You write the key questions and check whether the schema can represent their answers cleanly. For the clinic, typical questions include: listing appointments for a patient, listing a doctor's schedule for a day, and detecting double bookings. If these questions require awkward structures or duplicated facts, the design is usually wrong.

A second form of validation is traceability. You map each rule to the schema element that enforces it or supports enforcing it.

Example

Rule-to-schema traceability (clinic)

Rule	Design element(s)
R3	Appointment.patient_id is a mandatory reference to Patient
R4	Appointment.doctor_id is a mandatory reference to Doctor
R6	Constraint preventing duplicate (doctor_id, start_time) in Appointment

Traceability is not busywork. It is how you prove that your schema matches your intended meaning. If a rule has no corresponding design element, it is either missing from the model or being enforced elsewhere. Both cases should be explicit.

1.8 Common mistakes

The workflow fails in predictable ways.

The first mistake is to skip the story and jump straight to relations. This produces schemas that look tidy but do not match real rules.

The second mistake is to write vague rules like "a patient has appointments." Vague rules do not tell you whether appointments are mandatory, unique, or time-bound.

The third mistake is to treat validation as optional. If you do not test rules against the schema, you are guessing, not designing.

The fourth mistake is to add technical details too early. Performance and tooling matter later, but correctness must come first.

1.9 Mini checklist

Use this list before you move to the next chapter:

- Do you have a short story in plain language?
- Do you have 10–20 testable business rules?
- Are entities and relationships named and consistent?
- Do you have a compact schema with keys and references?
- Can you trace the most important rules to the schema?

Summary

Relational design becomes manageable when you treat it as a workflow. You start with a small story, force it into testable rules, structure it with a conceptual model, and translate it into a schema with keys and references. Finally, you validate the result by checking that rules are traceable and key questions are representable. This is how design becomes defensible, even before writing a single SQL statement.

Exercises

Answer in full sentences.

1. Write a short story (6–8 sentences) for a domain you know.
2. Extract 12 testable business rules from the story.
3. Classify each rule as identity, relationship, domain, or business constraint.
4. List the entities and relationships in plain language with cardinalities.
5. Write a first relational schema using the book notation.
6. Create a traceability map for at least 6 rules.
7. State two key questions your schema must answer and explain where the data lives.

Chapter 2

Core Concepts: Entities, Relationships, and Constraints

Learning objectives

In this chapter you will learn the core modelling vocabulary used throughout the book. You will be able to identify entities and attributes, describe relationships with cardinality and optionality, and express domain rules as constraints.

2.1 Entities and attributes

The first modelling decision is to separate *what exists* from *what is merely a property*. In relational design, this distinction matters because it determines where identity lives and where rules can be enforced.

An *entity* is something you need to refer to as a stable "thing". It persists across time, and you can talk about multiple instances of it. In a clinic domain, *Doctor* and *Patient* are entities because you need to track them independently of any single appointment. In a library domain, *Member*, *Title*, and *Copy* are entities for the same reason.

Definition

An *entity* is a thing with identity. An *attribute* is a property that describes an entity.

Attributes do not have independent identity inside the model. You do not "reference" a birth date or an email as a thing; you reference the patient or the member. That is why attributes belong *inside* an entity, while entities become relations later.

A practical test helps. If you remove the surrounding entity, does the concept still make sense as something you track and reference? If yes, it likely deserves entity status. If no, it is likely an attribute. This test is not perfect, but it prevents many beginner models from turning every noun into a table.

Principle

Model identity first. If you cannot name the thing you are modelling, you cannot reference it.

2.2 Relationships: describing how entities connect

Entities do not live in isolation. A relational database is mostly about capturing how entities connect, and how those connections constrain what data is valid.

A *relationship* expresses an association between entities. Some relationships are simple ("a patient has appointments"). Others carry meaning that must be preserved ("a member borrows a copy via a loan that has dates"). When a relationship carries its own meaning or properties, it often becomes a first-class structure later.

Definition

A *relationship* is an association between entities, described by:

- *cardinality*: how many instances can participate,
- *optionality*: whether participation can be absent,
- and sometimes *relationship attributes*: properties that belong to the association itself.

Even before drawing any diagram, you should be able to say relationships out loud in a precise sentence. For example: "Each appointment is with exactly one doctor and exactly one patient." This sentence already encodes a large part of the final schema.

2.3 Cardinality and optionality

Cardinality answers "how many". Optionality answers "must it exist". Together they define the shape of the model.

Consider the clinic. A patient can have many appointments over time. But for any given appointment, there is exactly one patient. This is a one-to-many relationship: Patient → Appointment.

The same logic applies to doctors. A doctor can have many appointments, but each appointment refers to exactly one doctor.

Optionality is the second dimension. A patient might exist without any appointments yet, which means the patient-to-appointments relationship is optional on the appointment side from the patient's perspective. But an appointment without a patient is typically invalid, so participation of patient in an appointment is mandatory.

You can express this informally with ranges:

$$0..N,\ 1..N,\ 0..1,\ 1..1$$

These ranges are not database syntax. They are a disciplined way to state meaning.

Example

Clinic relationship meaning
A patient may have zero or many appointments ($0..N$).
An appointment must have exactly one patient ($1..1$).
A doctor may have zero or many appointments ($0..N$).
An appointment must have exactly one doctor ($1..1$).

This level of precision is enough to translate to a correct relational design later. It also prevents a common mistake: modelling what is "typical" instead of what is "allowed". If the model says "a patient must have at least one appointment", you have introduced a rule that the real world usually does not have.

Example

University relationship meaning
A student may have zero or many enrollments (0..N).
An enrollment must have exactly one student (1..1).
A course may have zero or many enrollments (0..N).
An enrollment must have exactly one course (1..1).

2.4 Many-to-many relationships and why they matter

Many-to-many relationships are the most important ones to recognise early because they change the structure of the relational schema. They cannot be represented as a single foreign key on one side without losing information or creating duplication.

A classic example is "doctors have specialties". A doctor can have many specialties, and a specialty can apply to many doctors. If you try to store specialties as a single attribute on Doctor, you either restrict reality (one specialty only) or create repeating groups ("specialty1, specialty2, specialty3") which breaks basic relational structure.

In the library, the many-to-many shape appears in other ways, such as authors and titles if you model co-authors properly. Whenever you see many-to-many, you should anticipate an association structure later.

Example

Doctor $\leftrightarrow$ Specialty is many-to-many
One doctor can have many specialties.
One specialty can belong to many doctors.
This will later require a dedicated association relation.

2.5 Keys are not optional: identity is the backbone

You cannot reason about relationships without identity. If you cannot identify an entity instance, you cannot reference it, and you cannot enforce rules about it.

This is why every entity in a conceptual model must have a notion of a key, even if you do not decide the exact implementation yet. Sometimes a natural key exists (ISBN for a title). Sometimes it is better to use a surrogate key for stability and enforce natural uniqueness separately. The important point here is that identity must be explicit.

Principle

Do not confuse "unique in practice" with "unique by rule." Identity is a rule, not an observation.

2.6 Constraints: turning rules into enforceable structure

A model without constraints is a sketch. A relational database becomes reliable when constraints encode the rules of the domain. Constraints are not "extra". They are part of the meaning.

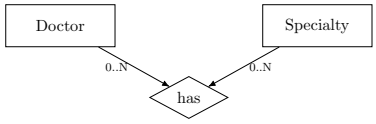


Figure 2.1: A many-to-many relationship requires an association structure to preserve meaning.

Definition

A *constraint* is a condition that valid data must satisfy. Constraints exist to prevent invalid states and preserve domain meaning.

It helps to classify constraints, because different constraints are enforced in different ways.

2.6.1 Entity integrity

Entity integrity is about identity. Primary keys must uniquely identify rows, and they must be present. If identity can be missing or duplicated, the model collapses.

2.6.2 Referential integrity

Referential integrity is about relationships. When one relation references another, the referenced thing must exist. This is how the database prevents “dangling references”, such as an appointment pointing to a patient that does not exist.

2.6.3 Domain constraints

Domain constraints restrict allowed values. A due date should not be before a checkout date. A status attribute should come from a defined set. Domain constraints are often the cheapest way to prevent silent corruption.

2.6.4 Business constraints

Business constraints are domain-specific rules that go beyond simple types and references. “No double booking” in the clinic is a business constraint. “So long as a copy has an active loan, it cannot have another active loan” is a business constraint in the library. These constraints are the ones people forget, and they are usually the ones that cause real incidents.

Example

Clinic constraint idea
Rule: “A doctor cannot have two appointments at the same start time.”
Design intention: prevent duplicate pairs (doctor_id, start_time) in appointments.

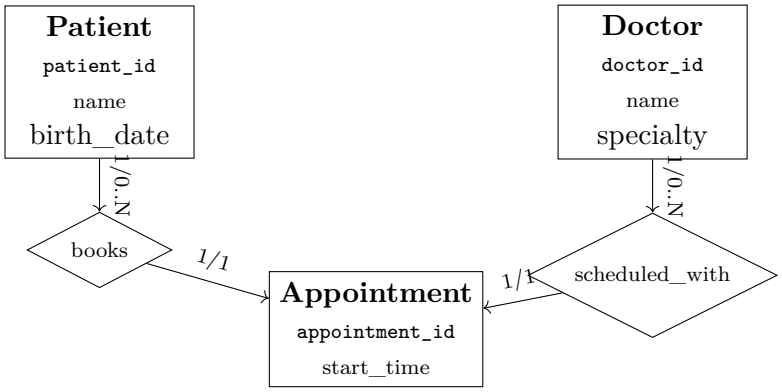
Note

Do not postpone constraints with the hope that “application logic will handle it.” Once invalid states exist, every future analysis becomes suspect.

2.7 A small ER-style diagram for communication

At this point, diagrams become useful. Not because diagrams are the goal, but because they force you to show identity, relationships, and constraints in one place. In this book, diagrams are used as communication devices: they help a reader see what the rules imply.

The diagram below is intentionally minimal. It is enough to talk about the model and to catch missing relationships early.



If you cannot draw a diagram like this without hesitation, it is usually a sign that the rules are still vague. In that case, you should return to the rule list and refine it. A diagram should be a consequence of clarity, not a substitute for it.

2.8 Common mistakes

The core vocabulary is simple, but mistakes are consistent.

The first mistake is treating attributes as entities. If a property has no independent identity, it should not be a relation.

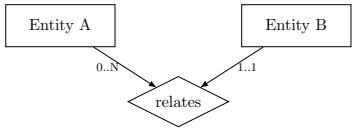


Figure 2.2: A minimal ER skeleton shows entities, a relationship, and directional participation.

The second mistake is ignoring optionality. If you do not specify it, your schema will usually allow invalid states.

The third mistake is collapsing many-to-many relationships. A list inside one entity is not a relationship; it is a modelling shortcut.

The fourth mistake is describing relationships without constraints. If you cannot state the rule as a testable sentence, it will not be enforced.

2.9 Mini checklist

Before you move to schema translation, check:

- Each entity has a clear identity rule.
- Every relationship has stated cardinality and optionality.
- Many-to-many relationships are recognized early.
- At least one business constraint is written for each key relationship.

Summary

Relational design starts with a clear conceptual vocabulary. Entities represent identity, attributes describe entities, and relationships capture associations with cardinality and optionality. Constraints are how rules become enforceable structure. If you can state the model precisely in words, you can translate it reliably into a relational schema later.

Exercises

Write a conceptual model in plain language for a small domain (library, gym, university, online shop). Do not draw yet.

1. List the entities and write one sentence for each explaining why it needs identity.
2. For each pair of entities that connect, write the relationship in a precise sentence.
3. For each relationship, state cardinality and optionality ($0..N, 1..1$, etc.).
4. Write 6 constraints: at least 2 referential, 2 domain, and 2 business constraints.
5. Identify one many-to-many relationship and explain why it cannot be stored as a single attribute.
6. Choose one attribute and argue why it should not be an entity.
7. Propose a minimal ER diagram with three entities and one relationship.

Chapter 3

Keys and Identity

Learning objectives

In this chapter you will learn how identity drives relational design. You will be able to choose appropriate keys, distinguish natural and surrogate identity, and design composite keys for association relations. You will also learn how key choices affect correctness, not just convenience.

3.1 Why identity comes first

Identity is the backbone of a relational model. If you cannot identify an entity instance, you cannot reference it, and constraints cannot bind it to anything else. All relationships depend on identity, and all constraints depend on references.

In practice, unclear identity produces duplicate rows that look different but represent the same thing. Once that happens, every rule is weakened. Two rows for the same patient mean two appointment histories, two sets of constraints, and two conflicting truths.

Principle

Identity is not optional. Every entity must have a stable way to be referenced.

3.2 Natural keys and surrogate keys

A *natural key* is a real-world identifier that already exists in the domain. An ISBN can identify a book title. A national health number might identify a patient. These keys can be meaningful, but they are not always stable or available.

A *surrogate key* is an identifier created for the system (often a numeric id). It carries no meaning, but it is stable and easy to reference. Many systems use surrogate keys for consistency and then enforce natural uniqueness separately.

Example

Library title identity

If ISBN is unique and stable in your domain, it can be a natural key:

$$\textbf{Title}(\underline{isbn}, title, author)$$

If ISBN is missing or unreliable, use a surrogate key and enforce uniqueness on ISBN when present.

Example

Clinic patient identity
Patients may not have a stable external identifier at first contact. A surrogate key avoids ambiguity:

$$\textbf{Patient}(\underline{patient\_id}, name, birth\_date)$$

Other identifiers can be stored as attributes with uniqueness constraints if required.

3.3 Key stability and lifecycle

Keys live through events: creation, correction, merges, and splits. If a key is not available at creation time, a surrogate key is usually safer. If a key can change, you must plan for updates across every reference.

3.3.1 Creation time and missing identifiers

Some identifiers arrive later. Clinic patients may receive a national identifier after the first visit. Library copies may receive a barcode after cataloguing. If you model these identifiers as primary keys, you will be forced to invent placeholders or delay record creation.

3.3.2 Merges and splits

Real entities can merge or split. Two patient records can be merged when a duplicate is discovered. A university course code can be split into two courses when curriculum changes. These events are manageable when identity is stable and references are consistent. They are chaotic when identity is tied to mutable attributes.

Example

Merging duplicate patients
If two rows represent the same person, you need to preserve references:

$$\textbf{Patient}(\underline{patient\_id}, name, birth\_date, national\_id? \text{ [unique]})$$

Merging uses a single `patient_id` while keeping optional external identifiers for matching.

3.4 Key quality criteria

Good keys share a few properties. These criteria are not formal rules, but they prevent most identity failures.

Principle

Key selection criteria
Prefer keys that are:

- stable over time,
- always available at creation time,
- minimal (no unnecessary attributes),
- and meaningful only if the meaning is truly stable.

If a key can change, you risk cascading updates to every reference. If a key is not available when the entity is created, you will be forced to invent placeholder values. Both outcomes degrade correctness.

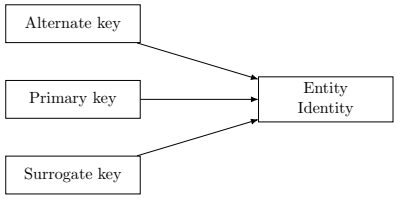


Figure 3.1: Primary, alternate, and surrogate keys all point to the same entity identity.

Note

Do not use names as primary keys unless the domain explicitly treats them as stable identifiers. Names change, collide, and create hidden duplicates.

3.5 Uniqueness beyond the primary key

Primary keys are not the only uniqueness rules. Many domains have alternate identifiers that must be unique when present. These are sometimes called alternate or candidate keys.

3.5.1 Alternate and candidate keys

A *candidate key* is any set of attributes that could serve as a primary key. Only one is chosen as the primary key, but the others still carry identity meaning.

3.5.2 Partial uniqueness

Some identifiers are optional but must be unique when present. This is common with email addresses, national IDs, or barcodes. The rule is not "always present" but "never duplicated."

Example

Unique-when-present

$$\textbf{Member}(\underline{member\_id}, name, email? \text{ [unique]})$$

Email is optional, but if two members have the same email, the identity model is broken.

3.6 Composite keys and association relations

Composite keys are common in association relations and in history tables. They combine two or more attributes to form identity.

The most important example is a many-to-many association. If a student can enroll in many courses, and a course can have many students, then the enrollment is identified by the pair.

Example

University enrollment identity

$$\begin{aligned}\textbf{Enrollment}(&\underline{student\_id} \to \textbf{Student},\\ &\underline{course\_id} \to \textbf{Course},\\ &enrolled\_at, grade?)\end{aligned}$$

The pair (student_id, course_id) is the identity of the enrollment.

Composite keys can also include a time attribute. If you track historical states, the identity might be (entity_id, effective_from). This keeps each historical record distinct.

3.7 Key availability in workflows

Key choice must fit how the system is used. In a clinic, you may need to create a patient record before any external identifier exists. In a library, a copy might be catalogued before its barcode is printed. These are workflow facts, not technical details.

Example

Copy identity in a library
If a barcode is assigned later, a surrogate key avoids placeholders:

$$\textbf{Copy}(\underline{copy\_id}, barcode \text{ [unique]}, status, title\_id \to \textbf{Title})$$

Barcode is unique when present, but not required for identity at creation time.

The same logic applies to university enrollment. The enrollment exists when the student registers, not when a final grade is assigned. So the grade must not be part of the key.

3.8 Choosing keys in clinic, library, and university

Key decisions become clearer when you compare domains.

Clinic: patients and doctors often need surrogate keys because external identifiers are inconsistent. Appointments typically use a surrogate key because multiple appointments can share time or participants.

Library: titles can sometimes use a natural key (ISBN), but copies almost never should. Loans and reservations are events; a surrogate key or a composite key with a timestamp both work, depending on how the domain defines uniqueness.

University: students often have an institutional id that is stable and can be a natural key. Courses often have a code, but if codes are reused across years, a surrogate key is safer. Enrollment is usually a composite key (student_id, course_id, term) if the domain allows repeat enrollment across terms.

3.9 Worked mini-example: university keys

A university model forces you to make identity choices early. Students often have a stable institutional id, but courses may have reused codes. Enrollment identity depends on the rule for retaking courses.

Example

University keys discussion

$$\begin{aligned}&\textbf{Student}(\underline{student\_id}, name, email?\ [\text{unique}])\\ &\textbf{Course}(\underline{course\_id}, code\ [\text{unique}], title)\\ &\textbf{Enrollment}(\underline{student\_id} \to \textbf{Student},\\ &\qquad \underline{course\_id} \to \textbf{Course},\\ &\qquad \underline{term}, status)\end{aligned}$$

If the same course code can be reused across years, `course_id` should be a surrogate key.

Example

Enrollment with term

If students can retake courses, include term in the identity:

$$\textbf{Enrollment}(\underline{student\_id} \to \textbf{Student}, \underline{course\_id} \to \textbf{Course}, \underline{term}, grade?)$$

This preserves the rule “one enrollment per student, course, and term.”

3.10 Common mistakes

Identity mistakes are easy to make and hard to fix later.

Mistake: Using email or name as a primary key.

Fix: Use a surrogate key and enforce email uniqueness when present.

Mistake: Using composite keys everywhere “for purity.”
Fix: Use composite keys only when the domain rule truly defines identity.
Mistake: Ignoring uniqueness of domain identifiers (barcode, national ID).
Fix: State alternate keys as explicit uniqueness constraints.
Mistake: Putting status or time attributes in the key.
Fix: Keep status and timestamps as non-key attributes unless the rule demands them.

3.11 Mini checklist

Before you finalize keys, check:

- Is every entity’s identity explicit and stable?
- Is the key available at creation time?
- If you use a natural key, is it truly unique by rule?
- Are alternate identifiers stated as uniqueness constraints?
- Do association relations have composite keys that match their uniqueness rules?
- Are time or status attributes excluded from keys unless the rule requires them?
- Can you explain how merges or splits would affect references?
- Are keys protected from changes in UI or workflow convenience?

Summary

Keys define identity and make relationships possible. Natural keys can be powerful when stable and universal, but surrogate keys often provide safer identity. Composite keys are common in association relations and history structures. Key choices must match the domain rules and the workflow that creates records. If identity is unclear, the rest of the schema cannot be trusted.

Exercises

Answer in full sentences and then write the corresponding schema fragments.

1. For a clinic, propose natural identifiers for Patient and Doctor and explain why they are or are not suitable as keys.
2. For a library, decide whether Title should use ISBN as a key and justify your choice.
3. Design a composite key for an association relation in a university domain and state the rule it enforces.
4. Give one example where a key attribute might change and explain the risk.
5. Propose a surrogate key strategy for a domain you know and list two benefits.
6. Show a relation where term or time belongs in the key, and explain the uniqueness rule.
7. Identify one attribute that should never be part of a key and justify why.
8. Write a uniqueness rule that is “unique when present” and show it in schema notation.
9. Describe a merge scenario (duplicates) and explain how your key choice makes it manageable.

In the next chapter we translate conceptual models into relational schemas. The mapping rules build directly on the identity and uniqueness choices made here.

Chapter 4

From Conceptual Model to Relational Schema

Learning objectives

In this chapter you will learn how to translate conceptual models into relational schemas in a disciplined way. You will be able to map entities and relationships to relations, choose foreign key placement correctly, and recognise when you need association relations. You will also learn how to write a schema specification that is meaningful even before SQL is introduced.

4.1 Translation as rule-based work

Once the domain is clear and identity is defined, moving to a relational schema should not feel like invention. It should feel like applying rules. This mindset matters because beginners often "design by improvisation", adding columns and tables until the model seems to work. That approach produces schemas that are hard to justify and even harder to maintain.

A relational schema is a statement about: what exists (relations), what uniquely identifies it (primary keys), how it connects (foreign keys), and what is allowed (constraints). The translation step is where meaning becomes enforceable structure.

4.1.1 Schema notation used in this book

We will specify relations without SQL, using a compact notation. For example:

$$\begin{aligned}\textbf{Appointment}(&\underline{\textit{appointment\_id}}, \textit{start\_time},\\ &\textit{patient\_id} \rightarrow \textbf{Patient},\\ &\textit{doctor\_id} \rightarrow \textbf{Doctor})\end{aligned}$$

Underline indicates the primary key. An arrow indicates a foreign key reference. Optional attributes may be written with a question mark (e.g., `return_date?`) when the idea of optionality matters.

This notation is intentionally simple. It is designed for clarity and discussion. It forces you to state identity and references explicitly.

4.2 The core mapping rules

The following rules cover most designs you will meet at introductory level. They are written as design rules, not as SQL steps.

Principle

Core mapping rules

1. Each entity becomes a relation.
2. Each relation has a primary key that represents identity.
3. A 1:N relationship becomes a foreign key on the N side.
4. A 1:1 relationship becomes a foreign key on one side, with a uniqueness constraint.
5. An N:M relationship becomes an association relation with two foreign keys.
6. Multivalued attributes become separate relations.
7. Attributes of a relationship belong in the structure that represents that relationship.

The final rule is often the most important. If a property describes the association, not either entity alone, then it belongs with the relationship. Dates on a loan are not properties of the member or the copy; they are properties of the borrowing event.

4.3 Mapping entities and relationships

4.3.1 Mapping entities: the easy part

Mapping an entity is straightforward. You create a relation and choose its primary key. The harder part is deciding which attributes are actually attributes, and which are hidden entities.

For example, in a clinic story, "specialty" might look like an attribute of Doctor. That can be fine for a minimal model. But if specialties have their own identity, descriptions, and governance (and doctors can have multiple specialties), then Specialty is an entity, not a string attribute. Translation works best when those conceptual decisions are made explicitly.

4.3.2 Mapping 1:N relationships: foreign keys and meaning

A one-to-many relationship means that one instance on the "one" side can be related to many on the "many" side. In the relational schema, this is represented by storing the primary key of the "one" side as a foreign key in the "many" side relation.

The important point is not the mechanical placement of the foreign key. The important point is that the foreign key expresses a constraint: each "many" row points to exactly one "one" row (if the relationship is mandatory).

Example

Clinic: Patient → Appointment is 1:N

Conceptual meaning: a patient can have many appointments; each appointment has one patient.

Relational schema:

$$\textbf{Patient}(\underline{patient\_id}, name, birth\_date)$$

$$\textbf{Appointment}(\underline{appointment\_id}, start\_time, patient\_id \rightarrow \textbf{Patient})$$

Optionality matters here. If an appointment must have a patient, then `patient_id` is mandatory. If an appointment could exist without a patient (usually not true), then `patient_id`

would be optional. This is not a technical detail. It is a statement about what data states are valid.

4.3.3 Mapping 1:1 relationships: choose a direction, then enforce it

One-to-one relationships require care. A 1:1 relationship means that each instance on either side is associated with at most one instance on the other side. In the relational schema, you can represent this with a foreign key on one side, but you must enforce uniqueness to maintain the one-to-one meaning.

Where you place the foreign key is a design choice guided by optionality and lifecycle. If one side can exist without the other, place the foreign key on the optional side. This avoids NULLs on the mandatory side and usually matches creation workflows.

Example

Example: Patient ↔ InsuranceProfile (1:1)

If a patient may have no insurance profile yet, then:

$$\textbf{Patient}(\underline{patient\_id}, \dots)$$

$$\begin{aligned}\textbf{InsuranceProfile}(&\underline{insurance\_profile\_id},\\ &patient\_id \to \textbf{Patient}\ [\text{unique}],\\ &provider, policy\_number)\end{aligned}$$

Uniqueness on `patient_id` in **InsuranceProfile** enforces the 1:1 relationship.

The main mistake is to store both foreign keys in both tables. That duplicates the relationship and creates inconsistency risk. A one-to-one relationship should be represented once, with the right constraint.

Principle

One-to-one placement rule
Place the foreign key on the optional side and enforce uniqueness there. This preserves the 1:1 meaning without introducing mandatory NULLs.

4.3.4 Mapping N:M relationships: association relations

Many-to-many relationships cannot be represented as a single foreign key without losing information. The relational model represents them using an association relation (also called a junction table). This relation contains at least two foreign keys, one to each participating entity. Its primary key is often the pair of foreign keys.

Example

Doctor ↔ Specialty (N:M)

$$\textbf{Doctor}(\underline{doctor\_id}, name)$$

$$\textbf{Specialty}(\underline{specialty\_id}, name)$$

$$\begin{aligned}\textbf{DoctorSpecialty}(&\underline{doctor\_id} \to \textbf{Doctor},\\ &\underline{specialty\_id} \to \textbf{Specialty})\end{aligned}$$

This structure becomes even more important when the relationship has attributes. For instance, if a doctor has a specialty *since a date*, that date belongs to the association relation:

$$\textbf{DoctorSpecialty}(\underline{doctor\_id}, \underline{specialty\_id}, since\_date)$$

This follows the principle: relationship attributes belong where the relationship is represented.

Note

If you add relationship attributes to an entity instead of the association relation, you silently change the meaning. Dates and roles are almost never properties of a single side.

4.3.5 Multivalued attributes are a hidden relationship

When you see an attribute that can take multiple values for a single entity, you are not looking at a single attribute. You are looking at a separate structure. In relational design, this becomes its own relation, typically with a foreign key back to the owning entity.

For example, if a member can have multiple phone numbers, "phone_numbers" is not an attribute in the relational sense. A separate relation is appropriate:

$$\textbf{MemberPhone}(\underline{member\_id} \rightarrow \textbf{Member}, \underline{phone\_number})$$

Whether `phone_number` is part of the primary key depends on the domain rules, but the modelling decision is the key point.

4.4 Relationship attributes and event entities

Some relationships are really events. If a relationship has time, status, or quantities, it usually deserves its own entity. This keeps the attributes where they belong and preserves history.

Example

Loan as event, not just a FK

Instead of storing a `current_copy_id` on **Member**, represent the borrowing as:

$$\textbf{Loan}(\underline{loan\_id}, checkout\_date, due\_date, return\_date?,$$
$$member\_id \rightarrow \textbf{Member}, copy\_id \rightarrow \textbf{Copy})$$

This captures the event and allows multiple loans over time.

4.5 Optionality mapping rules

Optionality should be decided explicitly. It changes where you place foreign keys and whether nulls are acceptable.

Rule A: If the relationship is mandatory on the "many" side, the FK is mandatory.
Rule B: If the relationship is optional, the FK may be optional.
Rule C: In 1:1, place the FK on the optional side.

Example

Optionality fragments

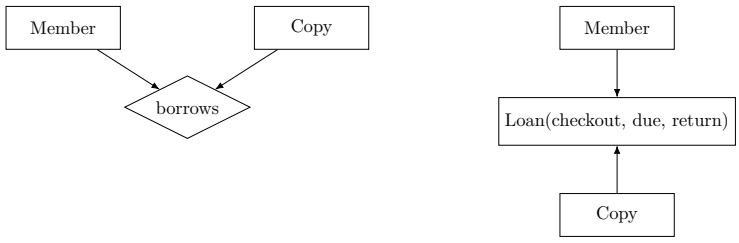


Figure 4.1: When a relationship has its own attributes, treat it as an event entity.

Mandatory 1:N:

$$\textbf{Appointment}(\underline{appointment\_id}, patient\_id \rightarrow \textbf{Patient})$$

Optional 1:N:

$$\textbf{Reservation}(\underline{reservation\_id}, member\_id? \rightarrow \textbf{Member})$$

Optional 1:1:

$$\textbf{InsuranceProfile}(\underline{insurance\_profile\_id}, patient\_id \rightarrow \textbf{Patient}\ [\text{unique}])$$

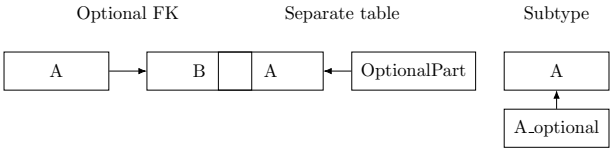


Figure 4.2: Three common ways to represent optional information in a relational schema.

4.6 A worked example: Library loans and reservations

The library domain is useful because it forces you to distinguish between a *Title* and a physical *Copy*. It also forces you to model events over time, such as loans, which are relationship-like structures with their own attributes.

Start with entities: Member, Title, Copy. A Title has many Copies (1:N). A Member borrows a Copy over time, which is best represented as a Loan entity because it carries dates and status.

The conceptual meaning "member borrows copy" is not a static relationship. It is a time-indexed event that must be recorded. That is why **Loan** exists.

Example

Library schema (logical, no SQL)

$$\textbf{Member}(\underline{member\_id}, name, email\ [\text{unique}])$$

$$\textbf{Title}(\underline{title\_id}, isbn\ [\text{unique}], title, author)$$

$$\textbf{Copy}(\underline{copy\_id}, barcode\ [\text{unique}], status, title\_id \rightarrow \textbf{Title})$$

$$\textbf{Loan}(\underline{loan\_id}, checkout\_date, due\_date, return\_date?, member\_id \rightarrow \textbf{Member}, copy\_id \rightarrow \textbf{Copy})$$

$$\textbf{Reservation}(\underline{reservation\_id}, request\_date, status, member\_id \rightarrow \textbf{Member}, title\_id \rightarrow \textbf{Title})$$

The important constraints are not cosmetic. They preserve meaning: a Copy belongs to exactly one Title, a Loan references exactly one Member and one Copy, and a Copy should not have two active loans at the same time. That last rule is not automatically guaranteed by foreign keys; it is a business constraint you must state and later enforce.

4.7 Mini-case: E-commerce orders

An order is an event with its own identity. Line items are not attributes of the order; they are entities that link products to orders with quantities.

Example

Order and order line (conceptual)

$$\textbf{Order}(\underline{order\_id}, placed\_at, customer\_id \rightarrow \textbf{Customer})$$

$$\textbf{OrderLine}(\underline{order\_id} \rightarrow \textbf{Order}, \underline{product\_id} \rightarrow \textbf{Product}, quantity, unit\_price)$$

The line is the association; quantities belong there, not on **Order**.

4.8 Common mistakes

There are recurring errors at this stage.

Mistake: Collapsing Title and Copy into a single relation.
Fix: Separate entities so availability and history can be represented cleanly.

Mistake: Treating time-indexed events as static relationships.
Fix: Model the event as its own relation (Loan, Appointment, Enrollment).
Mistake: Forgetting optionality.
Fix: Decide mandatory vs optional and place FKs accordingly.
Mistake: Putting relationship attributes on an entity.
Fix: Move dates, quantities, and roles to the association relation.

4.9 Checklist: mapping rules you can apply mechanically

Use these rules as a quick mechanical pass:

- Each entity becomes one relation.
- Each relation has a primary key that represents identity.
- 1:N becomes a foreign key on the N side.
- 1:1 becomes a foreign key with a uniqueness constraint.
- N:M becomes an association relation with two FKs.
- Multivalued attributes become their own relations.

4.10 Mini checklist

Before finalizing the relational schema, check:

- Does every relationship map to exactly one structure in the schema?
- Are all 1:1 relationships enforced with uniqueness, not duplication?
- Are relationship attributes stored on the association relation?
- Are multivalued attributes separated into their own relations?
- Are mandatory and optional references explicit?
- Can you explain why each foreign key is placed where it is?
- Are event entities used for time- or status-rich relationships?

Summary

Translating a conceptual model into a relational schema is a disciplined application of mapping rules. Entities become relations with keys. One-to-many relationships become foreign keys on the many side. Many-to-many relationships become association relations. If a relationship has attributes or time semantics, it usually needs its own structure. A schema is correct when it preserves meaning through identity, references, and constraints, even before SQL is introduced.

Exercises

Choose a domain (clinic, library, university, online shop) and answer in full sentences.

1. Identify one 1:N relationship and write the corresponding two relations in book notation.
2. Identify one N:M relationship and write the association relation.
3. Identify one relationship that has attributes (e.g., dates) and explain where those attributes belong.
4. Identify one multivalued attribute and rewrite it as a separate relation.
5. Write three business constraints that are not automatically enforced by foreign keys.
6. Show a 1:1 relationship and explain where you placed the foreign key and why.

7. Give one example where optionality changes the schema and explain the difference.
8. Sketch an order/order-line schema and justify where quantities and prices belong.
9. Write a constraint that enforces “one active loan per copy” in words.

In the next chapter we examine redundancy and normalization. The mapping choices here determine where redundancy appears and how it can be removed.

Chapter 5

Redundancy and Normalization

Learning objectives

In this chapter you will learn why redundancy is dangerous and how normalization helps. You will be able to recognise update, insert, and delete anomalies, explain the idea of functional dependency in plain language, and apply a minimal normalization toolkit (1NF, 2NF, 3NF) without turning the topic into algebra.

5.1 Why redundancy is a correctness problem

Redundancy looks harmless at first. You store the same fact in multiple places, and everything still works. The problem is that redundant facts do not remain consistent by accident. They remain consistent only if every future update is done perfectly. Real systems do not behave that way, especially when multiple people and services write data.

Normalization is not primarily about aesthetics. It is a way of designing schemas so that each fact has a clear home. When facts have a clear home, you reduce inconsistency and simplify enforcement of rules.

Definition

Redundancy is the repeated storage of the same fact in multiple places. A redundancy becomes harmful when it creates *anomalies*: situations where valid updates, inserts, or deletes produce invalid states or unintended data loss.

Principle

One fact, one home
If two rows can disagree about the same fact, the schema is already failing. Normalization is the discipline of giving each fact a single home.

5.2 How to spot redundancy early

Redundancy shows up in predictable ways. If you learn to recognise them early, you avoid large refactors later.

Repeated text fields are the first signal. If the same name, address, or description appears in many rows, ask which entity actually owns that fact.

Code+name duplication is another signal. If you store a code and its label together across many rows, you are likely repeating a fact that should live in its own relation.

Copy-paste facts are the most dangerous signal. If you find yourself copying attributes from one table to another “for convenience,” you are creating a future inconsistency.

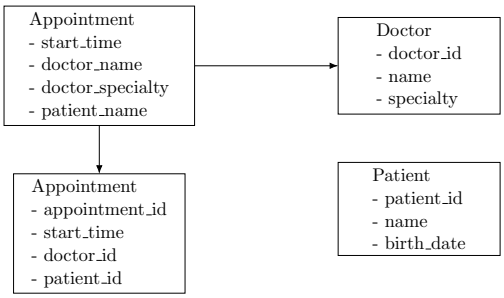


Figure 5.1: Normalization separates entity facts from event facts to remove redundancy.

5.3 The three classic anomalies

The anomaly triad is worth learning because it gives you a practical detection method.

An *update anomaly* happens when a fact appears in several rows and must be updated in all of them. If one row is missed, the database ends up containing contradictory facts.

An *insert anomaly* happens when you cannot insert one fact without inventing another. This often appears when a schema forces you to create a relationship record just to represent an entity that should exist independently.

A *delete anomaly* happens when deleting one record removes a fact you did not intend to remove. The schema couples facts that should be independent.

These anomalies are not rare edge cases. They happen naturally in schemas that mix "entity facts" with "event facts" in the same structure.

5.4 Normalization reasoning and worked examples

Normalization is easiest to understand when you start from something that looks like a spreadsheet. We will use informal dependencies, a minimal toolkit, and two worked examples to show the reasoning. Consider a naive attempt to record appointments:

Example

Bad design: mixed facts in one relation

$$\textbf{AppointmentRecord}(\underline{appointment\_id}, start\_time, doctor\_name, doctor\_specialty, patient\_name, patient\_birth\_date)$$

This table mixes appointment facts (start_time) with doctor facts (name, specialty) and patient facts (name, birth_date). As a result, the same doctor information will be repeated in every row for that doctor. The same patient information will be repeated in every row for that patient.

Now the anomalies appear immediately.

If a doctor changes specialty, you must update every appointment row for that doctor. That is an update anomaly. If you want to register a doctor before any appointments exist, you cannot do it without creating a fake appointment record. That is an insert anomaly. If you delete the only appointment a patient has, you may lose the only stored copy of that patient's details. That is a delete anomaly.

The table is not merely "not normalized". It makes correct operations fragile.

5.4.1 The informal idea of functional dependency

Normalization is often taught with formal definitions. You can do that later. For design work, you mainly need one idea: some attributes determine others.

If knowing one attribute always implies a specific value of another attribute, then the first attribute determines the second. For example, if patient_id uniquely identifies a patient, then patient_id determines patient_birth_date. In other words, patient facts should live where patient_id is the key.

Definition

An *informal functional dependency* means: "If you know X, you can determine Y." We write this as $X \to Y$.

In the bad appointment table, we implicitly claim:

$$appointment\_id \to doctor\_name,\ doctor\_specialty,\ patient\_name,\ patient\_birth\_date$$

But that is conceptually wrong. An appointment id should determine appointment details. Doctor details should be determined by doctor identity, and patient details by patient identity.

Normalization is the act of aligning "what determines what" with keys and relation boundaries.

Example

University: course and department facts

If each course belongs to one department, then department data should not be repeated in the course offering rows:

$$\textbf{Course}(\underline{course\_id}, code\ [\text{unique}], title, department\_id \rightarrow \textbf{Department})$$

$$\textbf{Department}(\underline{department\_id}, name, office\_location)$$

Course offerings can then reference **Course** without repeating department facts.

5.4.2 A minimal normalization toolkit

You do not need to memorise every normal form in the beginning. A minimal toolkit is enough to produce robust schemas in most student projects.

Principle

Normalize by dependency

If an attribute depends on only part of a key, or on another non-key attribute, it belongs elsewhere. Use dependencies to guide where facts live.

First Normal Form (1NF)

1NF is about representation. Attributes should be atomic, and relations should not contain repeating groups. If you store multiple values in one field, the relational model cannot constrain or query them reliably.

A common violation is a column like `phones = "912..., 934..., 965..."`. This is not a single attribute. It is a collection hidden inside a string.

The correction is not "split the string later". The correction is to represent the collection as a relation.

Example

From multivalued attribute to relation

Instead of:

$$\textbf{Member}(\underline{member\_id}, name, phones)$$

use:

$$\textbf{Member}(\underline{member\_id}, name)$$

$$\textbf{MemberPhone}(\underline{member\_id} \rightarrow \textbf{Member}, \underline{phone\_number})$$

Second Normal Form (2NF)

2NF matters when a relation has a composite primary key. A relation violates 2NF when a non-key attribute depends on only part of that composite key.

This often happens in association relations where someone incorrectly stores attributes of one side. For example, consider an association **DoctorSpecialty**(doctor_id, specialty_id). If you also store `doctor_name` in this relation, then doctor_name depends only on doctor_id, not on the pair. That creates redundancy and anomalies.

The correction is simple: move doctor attributes to **Doctor**.

Third Normal Form (3NF)

3NF is about avoiding non-key attributes determining other non-key attributes. A common pattern is storing a code and its description together in the same relation when the description is determined by the code.

For example, storing `specialty_name` inside **Doctor** when `specialty_id` exists and determines the name. That produces duplication and inconsistent spellings.

The correction is to create **Specialty** and reference it.

5.4.3 Clinic: normalized direction

A normalized direction is to separate doctor and patient facts from appointment facts.

Example

Normalized direction (conceptual)

$$\textbf{Doctor}(\underline{doctor\_id}, name, specialty)$$
$$\textbf{Patient}(\underline{patient\_id}, name, birth\_date)$$
$$\textbf{Appointment}(\underline{appointment\_id}, start\_time, doctor\_id \rightarrow \textbf{Doctor}, patient\_id \rightarrow \textbf{Patient})$$

Now each fact has a clear home. Doctor facts live in **Doctor**, determined by `doctor_id`. Patient facts live in **Patient**, determined by `patient_id`. Appointment facts live in **Appointment**, determined by `appointment_id`. Redundancy is reduced, and the anomalies largely disappear.

5.4.4 E-commerce: order lines mixing product facts

Order lines often mix product facts that should live elsewhere. Consider a naive **OrderLine** that stores product name and category in every row.

Example

Order line with duplicated product facts

$$\textbf{OrderLine}(\underline{order\_id} \rightarrow \textbf{Order}, \underline{product\_id}, product\_name, category, quantity, unit\_price)$$

If product_name changes, you must update every order line that references it. The fix is to separate product facts into **Product** and keep only the reference in **OrderLine**.

5.5 Normalization boundaries

Normalization can be taken too far, especially if taught as a checklist. In practice, schemas are sometimes denormalized deliberately for performance or simplicity, but that should be done with clear justification and with compensating controls.

Start by removing the anomalies you can see. If a change removes a real inconsistency risk, it is worth doing. If you cannot name the anomaly removed, the change is likely cosmetic.

In a first design pass, you should prefer clarity and correctness. If you later decide to denormalize, you should do it consciously and document which anomalies you are reintroducing and how you will manage them.

Note

A safe rule for early designs: normalize until the anomalies you can clearly see are removed. If you cannot explain what anomaly a transformation fixes, you may be refactoring for style rather than correctness.

5.6 Common mistakes

Normalization mistakes usually come from mixing levels of meaning.
Mistake: Treating event data as if it were entity data.
Fix: Move dates and statuses to the event relation (Loan, Appointment, OrderLine).
Mistake: Stopping after the first split without checking dependencies.
Fix: Ask what determines each attribute and move those that depend on other non-key attributes.
Mistake: Over-normalizing without a reason.
Fix: Name the anomaly you are removing; if you cannot, keep the structure.
Mistake: Duplicating code+name pairs across rows.
Fix: Create a separate lookup relation and reference it.

5.7 Mini checklist

Before you declare a schema "normalized enough," check:

- Does each fact have one clear relation where it is stored?
- Are multivalued attributes represented as separate relations?
- Do non-key attributes depend only on the full key?
- Are codes and their descriptions separated when a dependency exists?
- Can you explain which anomaly each split removes?
- Are event facts (dates, statuses, quantities) stored on event relations?
- Are you avoiding copy-paste attributes between relations?

Summary

Normalization is best understood as anomaly prevention. Redundancy creates update, insert, and delete anomalies, which eventually produce incorrect states. Functional dependency gives you the intuition for where facts belong: attributes should depend on the key and only the key. A minimal toolkit (1NF, 2NF, 3NF) is enough to produce robust schemas for most domains.

Exercises

Answer in full sentences.

1. Give one example of each anomaly (update, insert, delete) in a domain you know.
2. Show one example of a multivalued attribute and rewrite it as a separate relation (1NF).
3. Create a small association relation (N:M) and explain why storing an entity attribute inside it violates 2NF.
4. Describe one transitive dependency you might see in a real system and how you would eliminate it (3NF).
5. For a chosen domain, identify one place where you might denormalize later and what risks that would introduce.

6. For a clinic or library, sketch a table that mixes entity and event facts and explain the anomalies it creates.
7. For a university domain, list two attributes that should move to a separate relation and explain the dependency.

In the next chapter we move from redundancy to explicit constraints. Normalization reduces anomalies, but constraints make the remaining rules enforceable.

Chapter 6

Integrity Constraints and Correctness

Learning objectives

In this chapter you will learn how constraints make a relational design reliable. You will be able to classify constraints (entity, referential, domain, business), decide which rules can be enforced by the database, and document constraints as part of the schema specification.

6.1 Constraints are part of the meaning

A schema without constraints is only a storage layout. It can hold values, but it does not protect meaning. Relational databases become powerful because they are not passive containers: they can reject invalid states.

When a system relies only on application logic to maintain correctness, the database becomes an unsafe shared memory. Multiple services, scripts, and manual interventions will eventually bypass the intended rules. Constraints exist to prevent this drift.

Definition

A *constraint* is a condition that every valid database state must satisfy. Constraints exist to prevent invalid states and preserve domain meaning over time.

In this chapter we continue to avoid SQL. We focus on the conceptual role of constraints and how to specify them clearly, independent of implementation.

6.2 Constraint types with examples

Not all constraints are the same. Classifying them helps you know what the database can enforce directly and what may require additional mechanisms.

6.2.1 Entity integrity

Entity integrity is the requirement that identity is real. Primary keys must uniquely identify rows and must not be missing. This is not optional: without it, relationships cannot be trusted.

Even in a prose specification, you should state this clearly. For each relation, identity constraints include: the primary key attributes, whether they are mandatory, and whether they are stable.

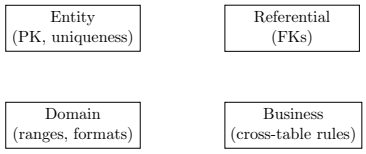


Figure 6.1: Four constraint categories clarify what can be enforced directly and what must be specified explicitly.

6.2.2 Referential integrity

Referential integrity is the requirement that references point to real things. If **Appointment** references **Patient**, then every appointment must point to an existing patient. Otherwise you are modelling relationships to non-existent entities, which breaks the semantics of the model.

Referential integrity also forces you to decide lifecycle behaviour. When a referenced entity is removed, what happens to referencing rows? Sometimes deletion is forbidden; sometimes references cascade; sometimes a record is archived instead of deleted. Even if you postpone the exact policy, you should recognise that it is part of correctness.

6.2.3 Domain constraints

Domain constraints restrict allowed values. They can be as simple as "age is non-negative", or as meaningful as "due date must be on or after checkout date". They often capture basic validity that is easy to enforce and easy to forget.

Domain constraints are valuable because they prevent silent corruption. If invalid values are allowed in, the system will later contain "impossible" states that break analysis and reporting.

6.2.4 Business constraints

Business constraints express domain-specific rules that go beyond identity, references, and simple value checks. These are the most important constraints from a correctness perspective, and also the most often missed.

A clinic rule like "a doctor cannot have two appointments at the same time" is a business constraint. A library rule like "a copy cannot have two active loans at the same time" is also a business constraint. These constraints define what the system *means*, not merely what it stores.

Example

University constraint mix

In a course registration model:

- Entity: Student.student_id is present and unique.
- Referential: Enrollment.student_id references Student.
- Domain: Enrollment.grade is one of the allowed grade symbols.
- Business: A student cannot enroll in the same course twice in the same term.

6.3 Feasibility of enforcement

A practical discipline is to treat each business rule as a constraint candidate. For each rule, you ask three questions.

First: can the database enforce it directly as a static constraint? Some rules map cleanly to uniqueness, not-null requirements, and foreign keys.

Second: if it cannot be enforced as a simple constraint, can it be enforced with a stronger mechanism? Some rules require temporal reasoning or overlap checks.

Third: if enforcement cannot realistically live in the database, can violations be detected reliably? A rule is still useful if you can detect when it is violated and react.

This discipline makes the design defensible. It also makes responsibilities explicit.

Principle

Rule → enforcement discipline

For each business rule:

- Can the database enforce it directly?
- If not, can it enforce it with a stronger mechanism?
- If not, how do we detect violations reliably?

Principle

Constraints are part of the schema
If a rule is not stated as a constraint, it is not part of the design. Write the rules before you choose where to enforce them.

There is a temptation to say "the database enforces everything". That is not always realistic. Some constraints involve complex temporal logic, external systems, or probabilistic rules. Others are policy decisions that evolve.

The right goal is not maximal enforcement at the database level. The right goal is: enforce what you can, and explicitly handle what you cannot. A rule that is unenforced and undocumented is the worst case. A rule that is enforced in the application but clearly specified and monitored can be acceptable.

Note

A practical rule: enforce identity and referential integrity in the database by default. For domain and business constraints, enforce in the database when feasible and document the rest with detection strategies.

6.4 Constraint catalogs

Catalogs make constraints visible. They also provide a checklist for validation later.

6.4.1 Clinic constraints

- C1: Patient.patient_id is mandatory and unique.
- C2: Doctor.doctor_id is mandatory and unique.
- C3: Appointment.appointment_id is mandatory and unique.
- C4: Appointment.patient_id references an existing Patient (mandatory).
- C5: Appointment.doctor_id references an existing Doctor (mandatory).
- C6: Appointment.start_time is mandatory.
- C7: Appointment.duration_min is mandatory and positive.
- C8: A doctor cannot have overlapping appointments.
- C9: A patient cannot have overlapping appointments with the same doctor.
- C10: Appointment.room_id references an existing Room when present.
- C11: A room cannot host overlapping appointments.
- C12: Appointment.status is one of {scheduled, completed, cancelled}.

6.4.2 Library constraints

- L1: Member.member_id is mandatory and unique.
- L2: Title.title_id is mandatory and unique.
- L3: Copy.copy_id is mandatory and unique.
- L4: Copy.title_id references an existing Title (mandatory).
- L5: Copy.barcode is unique when present.
- L6: Loan.loan_id is mandatory and unique.
- L7: Loan.member_id references an existing Member (mandatory).

- L8: Loan.copy_id references an existing Copy (mandatory).
- L9: Loan.due_date must be on or after Loan.checkout_date.
- L10: A copy has at most one active loan at any time.
- L11: Reservation.member_id references an existing Member (mandatory).
- L12: Reservations for a title are ordered by request_date.

6.5 Constraint writing style guide

Constraints should be written as testable sentences. A good constraint is precise, scoped, and observable.

Write constraints in the form: "For every X, condition Y holds." Avoid vague language like "usually" or "should".

Even without SQL, you should specify constraints as part of the schema. A reader should be able to answer: "what does this model allow?"

A compact documentation format works well:

- list relations and keys,
- list foreign key references,
- list uniqueness constraints,
- list domain and business constraints as numbered statements.

This style is close to how real systems are specified in design docs. It also makes your later SQL chapter easier to write: SQL becomes the translation of an already-clear specification.

Example

Constraint block example

For **Copy**:

- C1: Copy.copy_id is mandatory and unique.
- C2: Copy.title_id references an existing Title (mandatory).
- C3: Copy.barcode is unique when present.
- C4: Copy.status is one of {available, on_loan, retired}.

6.6 Common mistakes

Constraint mistakes are usually mistakes of silence.

Mistake: Failing to write the rule at all.
Fix: Record each rule as a numbered, testable constraint.

Mistake: Treating a business rule as a convenience.
Fix: Decide whether it is truly a rule; if yes, enforce or detect it.

Mistake: Hiding constraints in unrelated attributes.
Fix: State the constraint explicitly (e.g., uniqueness, no-overlap).

Mistake: Mixing domain and business constraints in prose only.
Fix: Separate value rules (domain) from relationship rules (business).

6.7 Mini checklist

Before you sign off a schema, check:

- Are identity and reference rules explicit for every relation?
- Are domain rules stated for key attributes with limited values?
- Are business rules written as constraints, not prose hints?
- Is each constraint placed where it can be enforced or detected?

- Are lifecycle rules (delete, archive) clearly described?
- Do constraint catalogs cover the critical entities and events?
- Are “active” and “overlap” defined unambiguously?

Summary

Constraints are not optional decorations; they are part of the meaning of a relational design. Entity and referential integrity provide the foundation for identity and relationships. Domain constraints prevent basic invalid values. Business constraints preserve domain-specific correctness, such as preventing double booking or concurrent active loans. A good design makes enforcement explicit, either directly in the database or through documented detection and control mechanisms.

Exercises

Answer in full sentences.

1. For a domain you know, write 8 business rules and classify each as entity, referential, domain, or business constraint.
2. Pick 3 of those rules and explain whether the database can enforce them directly, and why.
3. In a library model, propose a precise definition of “active loan” and rewrite the “one active loan per copy” rule using that definition.
4. In a clinic model, explain why interval-based appointments make “no double booking” harder than slot-based appointments.
5. Write a constraint specification block (C1, C2, ...) for one relation in your schema.
6. For a university model, define a business constraint that uses term or semester and state how it could be enforced.
7. Choose one rule that cannot be fully enforced by the database and describe how you would detect violations.
8. Write a domain constraint for a status field and explain what values are allowed.
9. Propose a constraint that prevents overlapping room bookings and state the needed attributes.

In the next chapter we validate designs without SQL. The constraint catalogs you wrote here will become inputs to validation.

Chapter 7

Design Validation Without SQL

Learning objectives

In this chapter you will learn how to validate a relational design before implementing it. You will be able to validate a schema by questions, by rule-to-schema traceability, and by controlled test data. You will also learn a practical review checklist that catches most modelling errors early.

7.1 Validation is part of design

A design is not correct because it looks reasonable. It is correct because it preserves meaning under use. Validation is the stage where you try to break the model on purpose, using the rules and the questions the system must support.

This book delays SQL, but validation does not require SQL. Validation is about logic, not syntax. A schema can be validated with careful reading, traceability, and small thought experiments.

Definition

Design validation is the process of checking that a schema:

- represents the intended domain meaning,
- supports the required questions,
- and prevents (or makes detectable) invalid states.

7.2 Validation artifacts as deliverables

Validation should leave artifacts you can review and reuse. These artifacts make the design auditable and easier to improve.

7.2.1 Questions list

Keep a short list of key questions that the system must answer. These questions drive the structure and keep validation grounded in use.

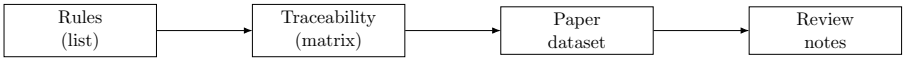


Figure 7.1: Validation artifacts connect rules to evidence and review notes.

7.2.2 Traceability map

Write a rule-to-schema table that links each rule to constraints or structures. This makes omissions visible.

7.2.3 Paper datasets

Create small, explicit datasets that include valid and invalid scenarios. They expose hidden ambiguity before implementation.

7.2.4 Review notes

Capture the decisions and open issues from validation. These notes become the starting point for the next review.

7.3 Method 1: Validate with key questions

Every domain has questions it must answer. If your schema cannot express the answers cleanly, the design is not finished.

The point is not to list every possible query. The point is to identify a small set of *key questions* that reflect the purpose of the system. These questions should include: questions about current state, historical state, and exceptional states.

A useful discipline is to write at least:

- 3 questions about lists and lookups,
- 3 questions about summaries and counts,
- 2 questions about detecting violations or anomalies.

Principle

Questions reveal structure
If a key question requires awkward joins or duplicated facts, the structure is wrong. Validate the schema by the questions it must answer.

7.3.1 Clinic: key questions

For the clinic domain, a minimal set could be:

- Q1: What appointments does a patient have this week?
- Q2: What is a doctor's schedule for a given day?
- Q3: Which patients have no future appointments booked?
- Q4: How many appointments did each doctor have last month?
- Q5: Are there any double bookings for a doctor?

Even without SQL, you can check if each question maps naturally to relations and relationships. If a question requires duplicated data or unnatural structures, the schema is signalling a problem.

7.3.2 Library: key questions

For the library domain:

- Q1: Which copies are available for a given title?
- Q2: What active loans does a member have?
- Q3: What is the loan history of a copy?
- Q4: Which loans are overdue today?
- Q5: Are there copies with more than one active loan?
- Q6: What is the reservation queue for a title?

Notice how these questions force the "title vs copy" separation. If you cannot answer Q1 and Q3 cleanly, your model is usually collapsing those concepts.

Example

University: key questions

- Q1: Which courses is a student enrolled in this term?
- Q2: Which students are on the waitlist for a course?
- Q3: Which courses have no enrolled students yet?
- Q4: Are there students enrolled without meeting prerequisites?

7.4 Method 2: Rule-to-schema traceability

Rules are the strongest validation tool because they are explicit. Traceability means that every important rule is linked to the design element that enforces it or supports it.

A traceability map is a short table. It turns vague assurance ("yes, we modelled that") into concrete evidence.

Principle

Traceability discipline

For each important rule:

- identify where it is enforced (constraint), or
- identify how it is supported (structure), or
- explicitly state that enforcement is external and how violations are detected.

Example

Clinic traceability example

Rule	Design element(s)
Appointment has exactly one patient	Appointment.patient_id is mandatory FK to Patient
Appointment has exactly one doctor	Appointment.doctor_id is mandatory FK to Doctor
No double booking (slot-based)	Unique(doctor_id, start_time) on Appointment

Example

Library traceability example

Rule	Design element(s)
Each Copy belongs to one Title	Copy.title_id is mandatory FK to Title
Each Loan references one Member	Loan.member_id is mandatory FK to Member
Each Loan references one Copy	Loan.copy_id is mandatory FK to Copy
Due date not before checkout date	Loan.due_date $\geq$ Loan.checkout_date
One active loan per copy	No two Loans with same copy_id and missing return_date
Reservation references a Title	Reservation.title_id is mandatory FK to Title
Reservation references a Member	Reservation.member_id is mandatory FK to Member
Copy barcode unique when present	Unique(Copy.barcode) when not null
Loan history preserved	Loan is an event relation, not a status field on Copy

Traceability also reveals missing design elements. If you have a rule and cannot point to any place in the design that supports it, the model is incomplete.

7.5 Method 3: Validate with controlled test data (on paper)

Even without SQL, you can validate by constructing small example datasets. This is an underrated technique. You propose a handful of rows for each relation and see whether:

- valid scenarios are representable without awkward duplication,
- invalid scenarios are prevented or at least visible as violations,
- history can be represented when time is involved.

This method is especially useful for business constraints.

7.5.1 Clinic thought experiment

Try to represent:

- a doctor with no appointments yet,
- a patient with two future appointments,
- an invalid double booking attempt.

If your design cannot represent the first two scenarios, it is too restrictive. If it cannot represent the invalid scenario as an explicit violation, it is too permissive.

Example

Clinic paper dataset (sketch)

Patient

patient_id	name	birth_date
P1	Ana Costa	1990-04-11
P2	Rui Silva	1985-09-03
P3	Marta Lima	2001-01-22

Doctor

doctor_id	name	specialty
D1	Joana Reis	cardiology
D2	Hugo Pereira	pediatrics
D3	Sara Pinto	dermatology

Appointment

appointment_id	start_time	patient_id	doctor_id
A1	2026-02-03 09:00	P1	D1
A2	2026-02-03 09:30	P2	D1
A3	2026-02-03 10:00	P3	D2

7.5.2 Library thought experiment

Try to represent:

- one title with two copies,
- one copy loaned and returned,
- one copy loaned to two members at the same time (invalid),
- two reservations for the same title.

A good design makes the invalid state either impossible (ideal) or immediately detectable. If the invalid state blends in without obvious contradiction, you need stronger constraints or clearer state modelling.

Example

Library test rows (sketch)

$$\textbf{Copy}(\underline{\textit{copy\_id}}, \textit{status}, \textit{title\_id} \rightarrow \textbf{Title})$$

Rows: (C1, on_loan, T1), (C2, available, T1)

$$\textbf{Loan}(\underline{\textit{loan\_id}}, \textit{copy\_id} \rightarrow \textbf{Copy}, \textit{member\_id} \rightarrow \textbf{Member}, \textit{return\_date}?)$$

Rows: (L1, C1, M1, missing), (L2, C1, M2, missing)
If both L1 and L2 are allowed, the “one active loan per copy” rule is violated.

Note

Keep test datasets small and explicit. Two or three rows per relation are usually enough to expose hidden ambiguity.

7.6 Common mistakes

Most beginner designs fail in repeatable ways. Validation methods catch them early.

7.6.1 Collapsing distinct concepts

Title vs copy, customer vs address, order vs payment, doctor vs specialty. If you collapse them, you will struggle to represent history, multiplicity, or policy. Key questions usually reveal this immediately.

7.6.2 Treating events as static relationships

Loans, appointments, payments, enrollments, subscriptions. These are events with time and state. If you model them as a single foreign key or a simple relationship, you lose history and constraints. Controlled test data exposes the problem fast.

7.6.3 Missing optionality decisions

If you do not decide what is mandatory, your schema quietly allows invalid states. Traceability catches this because rules like “exactly one” demand a mandatory reference.

7.6.4 Encoding collections inside attributes

Comma-separated lists, repeated columns, and JSON blobs used as a shortcut. These break 1NF and make constraints hard to express. Controlled test data exposes the awkwardness, and normalization principles give the fix.

7.7 Limits of validation

Validation improves correctness, but it cannot guarantee it. Some errors only appear under real data volume or evolving business rules.

Note

What validation cannot guarantee
Validation cannot prove a design is perfect. It can only reduce risk by exposing contradictions, missing rules, and weak assumptions.

7.8 A practical review checklist

Use this checklist to review any schema before implementation.

Example

Schema review checklist

- Identity: Does every entity have a clear primary key? Is it stable?

- Relationships: Are 1:N and N:M represented correctly (FK vs association relation)?
- Optionality: For each FK, is it mandatory or optional, and does that match the rules?
- Redundancy: Are you repeating the same facts across multiple relations?
- Time: Are time-based events represented as entities (Loan, Appointment) with their own attributes?
- Constraints: Are key business constraints stated and mapped to enforcement or detection?
- Questions: Can you answer the key questions naturally from the structure?

This checklist is short on purpose. If you apply it consistently, it catches most structural issues before a single line of SQL exists.

7.9 Mini checklist

Before moving to implementation, confirm:

- Each key question has a clear path through the schema.
- Every business rule is mapped to a constraint or detection plan.
- Test rows can represent both valid and invalid scenarios.

Summary

Validation is design work. You can validate a schema without SQL by checking whether it supports key questions, by building a rule-to-schema traceability map, and by constructing small controlled datasets to test representability and constraint behaviour. The goal is to ensure your model preserves meaning under real use, not only on paper.

Exercises

Pick one domain (clinic, library, university, online shop) and answer in full sentences.

1. Write 8 key questions (at least 2 about violations or anomalies).
2. Write 10 business rules and create a traceability table for at least 6 of them.
3. Propose a small dataset (3–5 rows per relation) that represents two valid scenarios.
4. Propose one invalid scenario and explain how your schema would prevent or detect it.
5. Use the checklist to find one improvement to your current model.
6. Identify one common mistake your schema might still contain and explain how you would test for it.
7. Add one question that forces a decision about optionality and explain how the schema answers it.
8. Write a short validation note that documents one assumption and one open issue.
9. Create a traceability row for a constraint that cannot be enforced directly and state how to detect violations.

In the next chapter we focus on naming and documentation. The artifacts you created here become inputs to those decisions.

Chapter 8

Naming, Documentation, and Design Hygiene

Learning objectives

In this chapter you will learn how naming and documentation affect correctness. You will be able to choose consistent names, document schemas at the right level of detail, and write small design records that make your modelling decisions auditable.

8.1 Why hygiene matters

A relational schema is a long-lived artifact. Even small projects accumulate time: new features, new people, new interpretations. When a schema is unclear, the database becomes a place where meaning drifts silently.

This is why naming and documentation are not cosmetic. They are part of correctness. If two developers interpret the same column differently, the system will store contradictory facts while still "working". Good hygiene reduces interpretability, and therefore reduces hidden bugs.

Definition

Design hygiene is the practice of keeping naming, structure, and documentation consistent so that the schema has a single stable interpretation.

8.2 Naming principles

Naming is about reducing ambiguity. A reader should guess the meaning of a relation or attribute correctly without needing a meeting.

Principle

Naming goals

Names should be:

- consistent across the schema,
- specific enough to avoid multiple interpretations,
- stable over time (avoid names tied to temporary implementation choices).

8.2.1 Entity names

Use singular nouns for entity relations: **Patient**, **Doctor**, **Appointment**, **Loan**. This matches the idea "a row represents one instance".

For association relations, use compound names that communicate meaning: **DoctorSpecialty**, **StudentCourseEnrollment**, **OrderLine**. Avoid generic names like **Mapping** or **Link** because they hide semantics.

8.2.2 Attribute names

Attribute names should communicate: what the value is, and what it refers to.

A reliable convention is:

- primary key: `<entity>_id` (e.g., `patient_id`),
- foreign key: same naming pattern, matching the referenced entity,
- booleans: `is_*`, `has_*` (e.g., `is_active`),
- timestamps/dates: include the semantic moment.
  Example: `checkout_date`, `return_date`, `created_at`.

Avoid names like `date` or `status` when multiple "dates" or "statuses" exist in the same domain. Ambiguous names force readers to search for meaning and increase the chance of misuse.

8.2.3 Do not encode meaning in magic abbreviations

Abbreviations can be useful, but they also hide meaning. If you use abbreviations, keep them standard and document them. Avoid project-specific abbreviations in the schema unless the domain itself uses them.

8.3 Naming conventions that scale

Small naming decisions become expensive when a schema grows. Use conventions that survive more tables, more teams, and more time.

8.3.1 Identifiers

Use `<entity>_id` for primary keys and match foreign keys to the referenced entity. This keeps joins readable and reduces misinterpretation.

8.3.2 Timestamps and dates

Name dates by their semantic moment: `created_at`, `checkout_date`, `valid_from`. Avoid generic `date` or `timestamp` in domains with multiple time concepts.

8.3.3 Booleans and state fields

Use `is_*` or `has_*` for booleans. For multi-state attributes, use a noun like `status` with a documented domain. If there are multiple statuses, name them explicitly (e.g., `loan_status`, `membership_status`).

8.4 Documentation layers

Documentation fails in two ways. It can be too small, leaving meaning undefined. Or it can be too large, becoming outdated.

Use layers so that each artifact has a clear purpose:

- README-level overview: domain narrative and goals.

- Schema specification: relations, keys, and constraints.
- Decision records: why modelling choices were made.

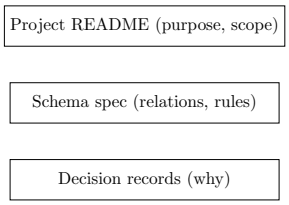


Figure 8.1: Layered documentation keeps purpose, structure, and decisions aligned.

This set is stable because it explains *meaning*, not implementation details.

Note

Avoid documenting things that will change frequently unless they are generated automatically. For example, detailed index plans or performance notes often drift. Meaning and constraints drift less, and are more valuable in early design.

8.5 Schema spec block you can paste into any project

A compact schema block prevents most confusion. It forces you to state purpose, identity, and constraints in a repeatable format.

Example

Schema spec block

Relation: **Appointment**
Purpose: Stores appointments scheduled between a patient and a doctor.
Primary key: `appointment_id` (surrogate).
Foreign keys: `patient_id` → **Patient** (mandatory), `doctor_id` → **Doctor** (mandatory).
Key constraints: start_time mandatory; no double booking per doctor at same start_time (slot-based).
Notes: If appointment duration varies, double booking becomes an interval overlap constraint.

Example

Constraint block template

C1: Appointment.patient_id references an existing Patient and is mandatory.
C2: Appointment.doctor_id references an existing Doctor and is mandatory.
C3: Appointment.start_time is mandatory.
C4: No two appointments may share the same (doctor_id, start_time) if time is slot-based.

When a constraint cannot be enforced directly, document how violations are detected. This keeps correctness visible even when enforcement is external.

8.6 Decision records: making modelling choices auditable

Some decisions are not obvious from the final schema. A decision record is a short note that explains why a choice was made. It is especially useful for identity choices, normalization boundaries, and state modelling.

A decision record should include:

- the decision,
- the alternatives considered,
- the rationale,
- and the implications.

Example

Decision record: Title vs Copy

Decision: Model **Title** and **Copy** as separate entities.
Alternatives: Single relation with one row per physical copy containing title metadata.
Rationale: Avoid duplication of title facts across copies; support availability and loan history; support multiple copies per title.
Implications: Loans reference copies; reservations reference titles; requires an "active loan" constraint per copy.

Example

Decision record: Surrogate vs natural key

Decision: Use a surrogate key for **Patient**.
Alternatives: Use national health number as primary key.
Rationale: Identifiers may be missing at first contact; surrogate keys keep identity stable across merges.
Implications: National health number stored as optional unique attribute; matching rules documented separately.

This format is useful in teaching because it shows that design is deliberate. It also mirrors how real teams justify schema decisions.

8.7 Common mistakes

Naming and documentation mistakes are easy to introduce.

Mistake: Using ambiguous attribute names (e.g., `date`, `status`).
Fix: Name attributes by their semantic role (`checkout_date`, `loan_status`).

Mistake: Mixing naming styles across the schema.
Fix: Choose one convention for identifiers and use it everywhere.

Mistake: Writing decision records only after problems appear.
Fix: Capture key decisions when you make them, even if brief.

Mistake: Documenting implementation details instead of meaning.
Fix: Prioritize rules, constraints, and identity.

8.8 Mini checklist

Before you share a schema, check:

- Are relation names singular and meaningful?
- Do identifiers follow a consistent pattern (`<entity>_id`)?
- Are date/time attributes named by their semantic moment?
- Are boolean attributes named with `is_*` or `has_*`?
- Is each multi-state field documented with allowed values?
- Are key decisions recorded in short decision records?
- Do schema blocks include purpose, keys, and constraints?
- Is documentation focused on meaning rather than implementation?

Summary

Design hygiene protects meaning. Consistent naming reduces ambiguity, and compact documentation makes modelling decisions stable over time. A good schema specification includes rules, constraints, and traceability, not just relation names. Decision records make the non-obvious choices auditable and teach readers how to reason about trade-offs.

Exercises

Answer in full sentences.

1. Choose a domain and propose naming conventions for relations and identifiers. Give three examples.

2. Write relation-level documentation blocks for two relations in your schema.

3. Write a constraint block (C1, C2, ...) for one relation with at least five constraints.
4. Write one decision record for a modelling decision that could be debated (e.g., surrogate vs natural key, denormalization boundary, state modelling).
5. Review your schema and rename at least three attributes to reduce ambiguity. Explain why.
6. Create a mini schema spec block for one relation in your project using the template above.
7. Identify two ambiguous names in a domain you know and propose better alternatives.
8. Write a documentation layer outline (README, schema spec, decision records) for a small project.

In the next chapter we apply these practices in a clinic case study. Naming and documentation choices will be part of the model, not an afterthought.

Chapter 9

Case Study: Clinic Scheduling (End-to-End)

Learning objectives

In this chapter you will apply the full workflow to a realistic clinic scheduling problem. You will learn how to move from narrative to business rules, from rules to a conceptual model, and from that model to a relational schema with constraints. You will also produce validation artifacts: key questions and a rule-to-schema traceability map.

9.1 Narrative

We will intentionally keep the domain small, but realistic enough to include common modelling decisions: identity, time, optionality, and a non-trivial business constraint.

Example

Clinic narrative
A clinic has doctors and patients. Patients book appointments with doctors. Each appointment has a start time and a planned duration. Appointments can be cancelled or rescheduled. Appointments take place in rooms. A doctor and a room cannot have overlapping scheduled appointments. Patients may exist in the system without any appointments yet.

The new elements here are duration, cancellation, rescheduling, and rooms. Those elements force us to think about time, state, and shared resources.

9.2 Step 1: Business rules

Write rules as testable statements. We will number them so they can be traced later.

Example

Business rules (clinic)
R1: A patient is uniquely identified by `patient_id`.
R2: A doctor is uniquely identified by `doctor_id`.
R3: An appointment is with exactly one patient.
R4: An appointment is with exactly one doctor.

R5: Every appointment has a start time.
R6: Every appointment has a planned duration (in minutes).
R7: An appointment has a status in {scheduled, cancelled}.
R8: A scheduled appointment is assigned to exactly one room.
R9: A doctor cannot have overlapping *scheduled* appointments.
R10: A room cannot have overlapping *scheduled* appointments.
R11: A patient can have zero or many appointments over time.
R12: A doctor can have zero or many appointments over time.

Rules R9 and R10 might look obvious, but they prevent an accidental modelling mistake: forcing every patient/doctor to have at least one appointment.

Rule R9 is the core business constraint. It is also a good example of a constraint that depends on time representation. Overlaps are interval logic, not simple uniqueness.

9.3 Step 2: Conceptual model

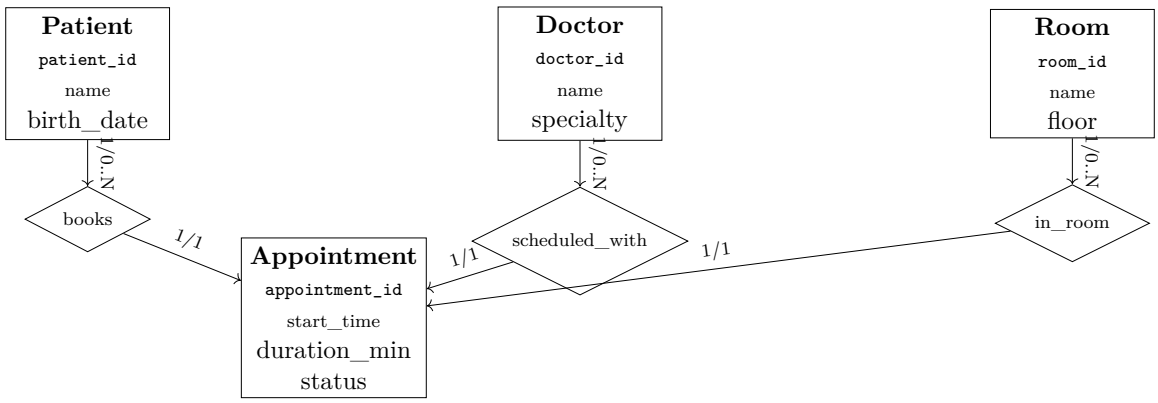
We will model three entities: Patient, Doctor, Appointment. Appointment is an entity rather than a mere relationship because it has its own attributes: start time, duration, and status.

9.3.1 Conceptual decisions

Before drawing anything, write the meaning in full sentences:

- Each appointment belongs to exactly one patient and exactly one doctor.
- A patient can have many appointments; a doctor can have many appointments.
- Appointment has a time interval defined by (start_time, duration).
- Scheduled appointments are assigned to a room; cancelled appointments do not require a room.
- Cancellation changes the enforcement of overlap rules: cancelled appointments do not block time.

9.3.2 ER-style diagram (minimal)



This diagram does not show the overlap constraint directly. That constraint is expressed as a business rule and later documented as a schema constraint.

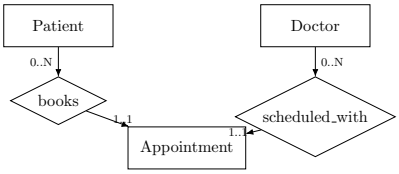


Figure 9.1: The clinic diagram highlights appointments as the event that connects patient and doctor.

9.4 Step 3: Relational schema (logical, no SQL)

Now we translate.

9.4.1 Relations

$$\textbf{Patient}(\underline{patient\_id}, name, birth\_date)$$

$$\textbf{Doctor}(\underline{doctor\_id}, name, specialty)$$

$$\textbf{Room}(\underline{room\_id}, name, floor)$$

$$\begin{aligned}\textbf{Appointment}(&\underline{appointment\_id}, start\_time, duration\_min, status,\\ &patient\_id \to \textbf{Patient}, doctor\_id \to \textbf{Doctor},\\ &room\_id? \to \textbf{Room})\end{aligned}$$

9.4.2 Attribute notes

We choose:

- `duration_min` as an integer, representing the planned duration.
- `status` as an attribute with a small allowed set.
- `room_id` as optional, but required when status is scheduled.

In a later implementation chapter, status could also be modelled as a reference to a **Status** lookup entity. For now, we keep it as a constrained attribute.

9.4.3 Constraints (conceptual specification)

Constraints are written as statements, independent of SQL.

Example

Constraints (clinic)
C1: Patient.patient_id is unique and mandatory.
C2: Doctor.doctor_id is unique and mandatory.
C3: Room.room_id is unique and mandatory.
C4: Appointment.patient_id references an existing Patient and is mandatory.
C5: Appointment.doctor_id references an existing Doctor and is mandatory.
C6: Appointment.room_id references an existing Room when status = scheduled.
C7: Appointment.start_time is mandatory.
C8: Appointment.duration_min is mandatory and positive.
C9: Appointment.status is in {scheduled, cancelled}.
C10: For each doctor, scheduled appointments must not overlap in time.
C11: For each room, scheduled appointments must not overlap in time.

Constraint C9 is the key business constraint. It is not a simple uniqueness constraint when time is interval-based. It requires overlap detection between time intervals of the same doctor for appointments with status = scheduled.

9.5 Step 5: Validation without SQL

9.5.1 Key questions

These questions are chosen to test whether the schema supports essential operations.

- Q1: What appointments does a patient have in a given week?
- Q2: What is a doctor's schedule on a given day?
- Q3: Which patients have no future scheduled appointments?
- Q4: How many appointments did each doctor complete (scheduled, non-cancelled) in a month?
- Q5: Are there any overlaps among scheduled appointments for the same doctor?
- Q6: Which rooms are in use at a given time?
- Q7: Which appointments were cancelled or rescheduled this week?

If the schema cannot express these questions without duplication or unnatural structures, the model needs revision.

9.5.2 Rule-to-schema traceability

Now we justify that rules are represented.

Example

Traceability map (clinic)

Rule	Design element(s)
R1	Patient.patient_id unique and mandatory (C1)
R2	Doctor.doctor_id unique and mandatory (C2)
R3, R4	Appointment has mandatory FKs to Patient and Doctor (C4, C5)
R5, R6	start_time and duration_min mandatory (C7, C8)
R7	status domain constraint (C9)
R8	scheduled appointments require room assignment (C6)
R9	overlap prevention for scheduled appointments (C10)
R10	room overlap prevention (C11)
R11, R12	1:N implied by FK direction and lack of uniqueness on FKs

9.5.3 Controlled test data (paper test)

Try to represent these scenarios:

- One doctor, two appointments on the same day, non-overlapping.
- A cancelled appointment that overlaps a scheduled one (should be valid).
- Two scheduled appointments that overlap (should be invalid by C10).

If you cannot express the invalid scenario clearly as a violation, the model is too permissive. If you cannot express the cancelled-overlap scenario, the model is too restrictive.

9.5.4 Invalid scenario dataset

Example

Invalid dataset: overlapping room usage

appointment_id	start_time	duration_min	doctor_id	room_id
A10	2026-02-05 09:00	60	D1	R1
A11	2026-02-05 09:30	30	D2	R1

Appointments A10 and A11 overlap in room R1. This violates C11 (no overlapping scheduled appointments per room).

9.6 Time model and rescheduling

We use intervals defined by (`start_time`, `duration_min`). This fits real clinics where appointment lengths vary. It also makes overlap constraints explicit.

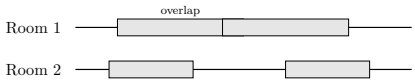


Figure 9.2: Overlapping appointments in the same room violate the room scheduling rule.

9.6.1 Discrete slots vs intervals

If the clinic uses discrete slots (e.g., 15-minute blocks and fixed duration), then the overlap rule can be expressed as a simple uniqueness constraint on (doctor_id, start_time). That is easier to enforce.

Intervals are more expressive, but enforcement requires overlap logic. The modelling lesson is that representation choices affect which constraints are simple and which are complex.

9.6.2 Rescheduling and cancellation

Rescheduling is treated as changing `start_time` and `duration_min`. We do not keep historical changes in this model. If history is required, a separate **AppointmentChange** entity would be appropriate.

Cancellation keeps the appointment but changes its status. Cancelled appointments do not block time or room usage.

9.7 Common mistakes

Clinic models fail in predictable ways.

Mistake: Treating room as an attribute instead of an entity.
Fix: Model **Room** with its own identity and constraints.

Mistake: Using status to avoid overlap rules.
Fix: Keep overlap constraints and decide which statuses are excluded explicitly.

Mistake: Storing end time and duration inconsistently.
Fix: Choose one representation and derive the other if needed.

Mistake: Forgetting rescheduling rules.
Fix: Specify whether rescheduling overwrites or creates new records.

9.8 Mini checklist

Before you finalize the clinic model, check:

- Are appointments modelled as event entities with their own identity?
- Are doctor and room overlap rules explicit and testable?
- Is the time representation consistent (start_time + duration)?
- Are cancellations represented as a status with clear semantics?
- Are optional room assignments limited to cancelled appointments?
- Do key questions cover schedules, rooms, and cancellations?
- Is traceability complete for all business rules?

Summary

This case study shows the full workflow in a small domain. The model is built from a narrative, made precise through business rules, translated through disciplined mapping, and protected through constraints. Validation is demonstrated using key questions, traceability, and paper test scenarios.

Exercises

Extend this clinic model. Answer in full sentences, then update the schema in book notation.

1. Add *Room* and the rule: “A room cannot host overlapping scheduled appointments.”

2. Add a rule limiting daily appointments per doctor (choose a number and justify it).
3. Add patient contact details and decide which are unique (if any).
4. Add an appointment “type” (consultation, exam, follow-up) and decide how to model it (attribute vs entity).
5. Update the traceability map for your new rules.
6. Define an invalid scenario dataset that violates a room overlap constraint and explain the violation.
7. Write a constraint for rescheduling that prevents moving an appointment into a blocked time.
8. Add a rule about room availability (maintenance or closure) and show where it belongs in the schema.

In the next chapter we apply the same workflow to a library case study. Notice how different events (loans, reservations, fines) change the structure and constraints.

Chapter 10

Case Study: Library System (End-to-End)

Learning objectives

In this chapter you will apply the full workflow to a library domain. You will learn how to model titles versus physical copies, how to represent time-based events like loans, and how to encode key constraints such as "one active loan per copy". You will also practise validation with questions, traceability, and controlled test data.

10.1 Narrative

The library domain is a strong teaching example because it forces two modelling decisions that appear in many real systems: distinguishing an abstract item from its physical instances, and modelling events over time.

Example

Library narrative
A library registers members. The library stores book titles, and each title can have multiple physical copies. Copies belong to a branch (a physical location). Members borrow copies through loans, with checkout and due dates. A copy may be returned later. A copy cannot be on loan to two members at the same time. Members can also place reservations for titles. Reservations are handled in first-come-first-served order. Overdue loans can generate fines, and fines can be paid.

The phrase "title can have multiple copies" is the central constraint. If you collapse title and copy, the model becomes confusing and will produce anomalies.

10.2 Step 1: Business rules

We write rules as testable statements. We keep them focused on meaning and constraints.

Example

Business rules (library)
R1: A member is uniquely identified by `member_id`.
R2: A title is uniquely identified by `title_id`.

R3: A copy is uniquely identified by `copy_id`.
R4: Each copy belongs to exactly one title.
R5: A title can have zero or many copies.
R6: Each copy belongs to exactly one branch.
R7: A loan is for exactly one member and exactly one copy.
R8: A loan has a checkout date and a due date.
R9: A loan may have a return date (if returned).
R10: A copy has at most one active loan at a time.
R11: A reservation is made by exactly one member for exactly one title.
R12: Reservations are served in first-come-first-served order by request date.
R13: A member can have zero or many reservations over time.
R14: A title can have zero or many reservations over time.
R15: An overdue loan can generate a fine for the member.
R16: A fine can have zero or many payments.

Rule R9 is the core business constraint. It prevents an impossible state that would otherwise be representable.

10.3 Step 2: Conceptual model

We model the following entities: Member, Title, Copy, Branch, Loan, Reservation, Fine, Payment. Loan and Reservation are entities because they carry their own meaning and attributes. They are not mere relationships.

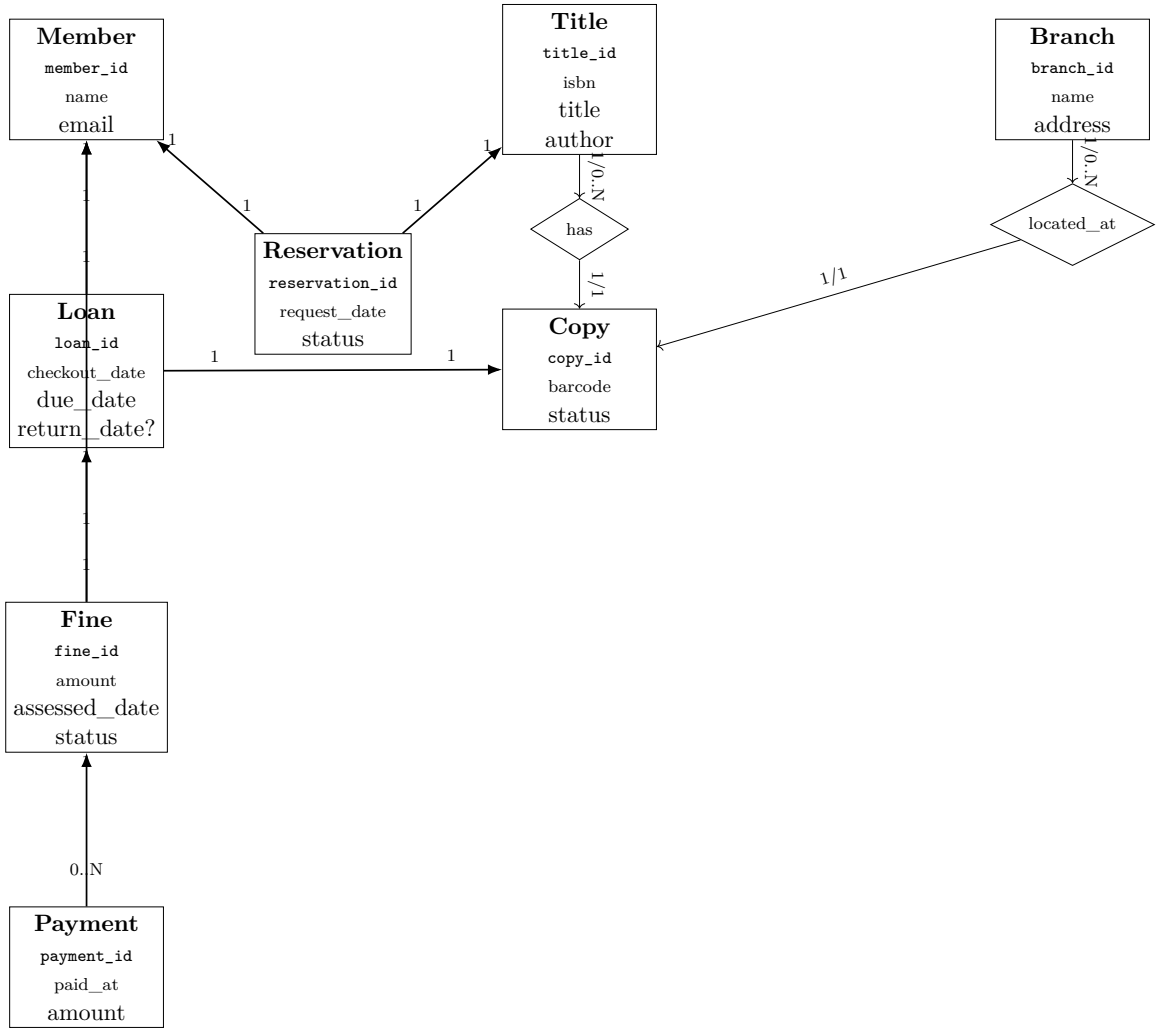
10.3.1 Conceptual decisions

Write the meaning in sentences:

- A title is an abstract bibliographic item; a copy is a physical instance.
- Copies belong to exactly one title; titles can have many copies.
- Copies belong to exactly one branch.
- Loans represent borrowing events over time and must be recorded historically.
- A reservation is for a title (not for a specific copy), because copies may not be available yet.
- Fines are linked to overdue loans; payments are linked to fines.

These decisions are what make the model coherent.

10.3.2 ER-style diagram (minimal)



The diagram is intentionally simple. The "one active loan per copy" constraint is not drawn; it is specified as a constraint.

10.4 Step 3: Relational schema (logical, no SQL)

Now we translate meaning into relations with keys and references.

$$\textbf{Member}(\underline{member\_id}, name, email)$$

$$\textbf{Title}(\underline{title\_id}, isbn, title, author)$$

$$\textbf{Branch}(\underline{branch\_id}, name, address)$$

$$\textbf{Copy}(\underline{copy\_id}, barcode, status, title\_id \rightarrow \textbf{Title}, branch\_id \rightarrow \textbf{Branch})$$

$$\textbf{Loan}(\underline{loan\_id}, checkout\_date, due\_date, return\_date?, member\_id \rightarrow \textbf{Member}, copy\_id \rightarrow \textbf{Copy})$$

$$\textbf{Reservation}(\underline{reservation\_id}, request\_date, status, member\_id \rightarrow \textbf{Member}, title\_id \rightarrow \textbf{Title})$$

$$\textbf{Fine}(\underline{fine\_id}, amount, assessed\_date, status, member\_id \rightarrow \textbf{Member}, loan\_id \rightarrow \textbf{Loan})$$

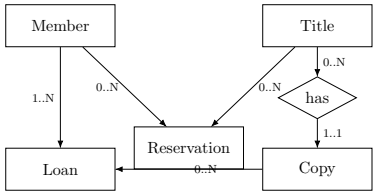


Figure 10.1: The library diagram separates titles from copies and models loans and reservations as events.

$$\textbf{Payment}(\underline{payment\_id}, paid\_at, amount, \\ fine\_id \rightarrow \textbf{Fine})$$

10.4.1 Attribute notes

Two attributes deserve explicit meaning:

- `Copy.status` expresses the library's view of a copy (available, lost, repair, ...).
- `Reservation.status` expresses the state of a reservation (active, fulfilled, cancelled, ...).
- `Fine.status` expresses the state of a fine (open, paid, waived, ...).

In later chapters you may model these as reference entities. For now, we keep them as constrained domains.

10.4.2 Constraints (conceptual specification)

We now specify the constraints that preserve meaning.

Example

Constraints (library)
C1: Member.member_id is unique and mandatory.
C2: Title.title_id is unique and mandatory.
C3: Copy.copy_id is unique and mandatory.
C4: Branch.branch_id is unique and mandatory.
C5: Copy.title_id references an existing Title and is mandatory.
C6: Copy.branch_id references an existing Branch and is mandatory.
C7: Loan.member_id references an existing Member and is mandatory.
C8: Loan.copy_id references an existing Copy and is mandatory.
C9: Loan.due_date is on or after Loan.checkout_date.
C10: A Copy has at most one active Loan at a time.
(Active loan = loan with return_date missing, or status = active.)
C11: Reservation.member_id references an existing Member and is mandatory.
C12: Reservation.title_id references an existing Title and is mandatory.
C13: Reservations for a title are served by ascending request_date.
C14: Fine.loan_id references an existing Loan and is mandatory.
C15: Payment.fine_id references an existing Fine and is mandatory.

Constraint C10 is the key business constraint. It is the one that protects the meaning "a copy cannot be loaned to two people at the same time". The definition of "active" must be explicit for this constraint to be enforceable or detectable.

In this model, a loan is active when `return_date` is missing (or status = active, if you model status). This definition should appear in the constraint catalog and in validation scenarios.

10.5 Branches, fines, and queue policy

These extensions change the operational meaning of the system. They also introduce new constraints that must be explicit.

10.5.1 Branches

Branches are physical locations. Copies belong to one branch, and availability is branch-specific. This matters for questions like "which branch has an available copy?"

10.5.2 Fines and payments

Fines are linked to overdue loans and to the member responsible. Payments are linked to fines and can be partial. This keeps money flow separate from loan events while maintaining traceability.

10.5.3 Reservation queue

Reservations are served in first-come-first-served order by request date. This is a policy rule, not a database default. It must be written as a constraint and reflected in validation.

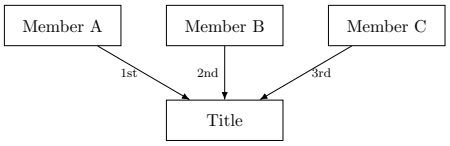


Figure 10.2: Reservation queues preserve first-come-first-served order for each title.

10.6 Step 5: Validation without SQL

10.6.1 Key questions

Choose questions that test the purpose of the system and the central constraints.

- Q1: Which copies are available for a given title?
- Q2: What active loans does a member have?
- Q3: What is the loan history of a copy?
- Q4: Which loans are overdue today?
- Q5: Are there any copies with more than one active loan?
- Q6: What is the reservation queue for a title?
- Q7: Which branch holds the available copies of a title?
- Q8: What fines are outstanding for a member?

The schema supports these questions naturally: availability comes from copies and active loans, history comes from the loan records, and reservations connect members to titles.

10.6.2 Rule-to-schema traceability

We link rules to design elements.

Example

Traceability map (library)

Rule	Design element(s)
R4, R5	Copy.title_id FK to Title expresses 1:N (C5)
R6	Copy.branch_id FK to Branch expresses 1:N (C6)
R7	Loan has mandatory FKs to Member and Copy (C7, C8)
R8, R9	Loan has checkout/due dates and optional return_date (C9)
R10	One active loan per copy (C10)
R11	Reservation has mandatory FKs to Member and Title (C11, C12)
R12	Reservation queue order by request_date (C13)
R13, R14	Multiple reservations possible because no uniqueness on those FKs
R15	Fine references Loan and Member (C14)
R16	Payment references Fine (C15)

10.6.3 Controlled test data (paper test)

Construct these scenarios:

- A title with two copies, one available and one on loan.
- A copy loaned and returned, creating history.
- Two loans for the same copy with missing return_date (invalid under C10).
- Two reservations for the same title (valid).

If the invalid state is not clearly representable as a violation, you need stronger constraint specification. If you cannot represent history cleanly, you likely collapsed events into static relationships.

10.6.4 Paper dataset: valid scenario

Example

Valid dataset (excerpt)

copy_id	title_id	branch_id	status
C1	T1	B1	available
C2	T1	B1	on_loan

loan_id	copy_id	member_id	checkout_date	return_date
L1	C2	M1	2026-02-01	missing
L2	C2	M1	2025-12-10	2025-12-20

10.6.5 Paper dataset: invalid scenario

Example

Invalid dataset (violates C10)

loan_id	copy_id	member_id	checkout_date	return_date
L3	C1	M2	2026-02-03	missing
L4	C1	M3	2026-02-04	missing

Both loans are active for the same copy, violating the “one active loan per copy” rule.

10.7 Design note: why reservations are for titles

Some readers try to reserve a copy. That seems concrete, but it is usually wrong at the conceptual level. A reservation is a promise about future availability, and the library typically fulfils it with any available copy of the title.

Modelling reservations for titles also matches real operations: a title can have multiple copies, and copies can be lost, repaired, or replaced. If reservations are tied to a specific copy, the system becomes brittle.

This is a general modelling lesson: when a concept is a policy about a class of items, it should reference the abstract entity, not a specific instance.

10.8 Common mistakes

Library models often fail in familiar ways.

Mistake: Collapsing Title and Copy.
Fix: Separate entities so availability and history are clean.

Mistake: Treating reservations as copy-level.
Fix: Reserve the title and allocate a copy when available.

Mistake: Leaving “active loan” undefined.
Fix: Define active explicitly (e.g., return_date missing) and enforce it.

Mistake: Mixing fines into the loan table.
Fix: Keep fines as a separate entity linked to loans and members.

10.9 Mini checklist

Before finalizing the library model, check:

- Are Title and Copy separated with a clear 1:N relationship?
- Is “active loan” defined and used consistently?
- Does each copy belong to one branch?
- Are reservation queues ordered by request date?
- Are fines and payments modelled as separate entities?
- Can the schema represent loan history without duplication?
- Do constraints cover availability and queue policy?

Summary

This case study demonstrates the full workflow in a library domain. The key modelling decision is separating titles from copies, and representing borrowing as a time-based event via loans. Constraints, especially “one active loan per copy”, preserve correctness. Validation through questions, traceability, and paper tests makes the design defensible before implementation.

Exercises

Extend the library model. Answer in full sentences, then update the schema in book notation.

1. Add support for multiple authors per title. Decide whether Author is an entity and model it.
2. Add fines for overdue loans. Decide what facts should be stored versus derived.
3. Add a branch concept (multiple library locations) and decide where copies belong.
4. Add a rule limiting maximum active loans per member and describe how it would be enforced or detected.
5. Update the traceability map for your new rules and constraints.
6. Define “active loan” precisely and write the corresponding constraint in words.
7. Create a valid and an invalid paper dataset for reservations and explain the queue policy.
8. Add payments for fines and list two constraints that protect correctness.

In the next chapter we focus on state and time. Loan history, reservation queues, and fine status will connect directly to those concepts.

Chapter 11

Modelling State and Time

Learning objectives

In this chapter you will learn how to model concepts that evolve over time. You will be able to distinguish entities from events, choose between storing state versus deriving it, and express time-based business constraints clearly. You will also learn common patterns for modelling status, history, and temporal correctness.

11.1 Why time changes everything

Many beginner schemas look correct until time enters the picture. Once you need history, active versus inactive states, or "what was true at a past moment", naive structures stop working.

Time introduces two challenges. First, facts can change: a member changes email, a copy changes status, an appointment gets cancelled. Second, the system must preserve history: you want to know what happened, not just what is true right now.

Relational design remains stable under time if you decide, explicitly, what should be stored as a changing attribute and what should be represented as a sequence of events.

Definition

A *state* is a property of an entity at a point in time (e.g., copy is available). An *event* is a record of something that happened at a time (e.g., a loan checkout).

11.2 Choosing authoritative truth

When both events and current state are stored, one must be authoritative. If the event stream says "returned" but the state says "on loan," the model is undefined.

In a library, the authoritative truth might be the event stream of loans. In a clinic, the authoritative truth might be the current appointment schedule. The choice depends on what you must audit and how often state changes.

11.3 Entities versus events

A reliable modelling decision is to treat "things" as entities and "happenings" as events. Loans and appointments are not static links; they are happenings with time and attributes. This is why **Loan** and **Appointment** are typically entities in a relational schema, even though they connect two other entities.

If you model an event as a static relationship, you often lose:

- history (you overwrite old values),
- constraint clarity (active state becomes unclear),
- and auditability (you cannot explain how you reached the current state).

Example

Bad idea: store current borrower on Copy

$$\textbf{Copy}(\underline{copy\_id}, title\_id \to \textbf{Title}, current\_member\_id?)$$

This loses history and makes "when was it borrowed?" hard to represent. A **Loan** event is a cleaner model.

11.4 Storing state versus deriving state

A design often has two options.

Option A: store the state directly (e.g., `Copy.status = available/on_loan/lost`).

Option B: derive the state from event history (e.g., "a copy is on loan if it has an active loan").

Both options can be correct. The choice depends on how the domain behaves and what needs to be enforced.

11.4.1 When storing state helps

Storing state is useful when:

- state changes are not always implied by one event stream,
- you need explicit human decisions (e.g., marking a copy as lost),
- you want fast access to current state and you can enforce consistency.

11.4.2 When deriving state helps

Deriving state is useful when:

- "current state" is fully implied by event history,
- you want a single source of truth (events),
- correctness depends on history (auditing, analytics).

In many real designs, you combine both: events as the source of truth, with a stored current-state attribute as a cache. If you do that, you must define consistency rules clearly, otherwise you create contradictory truths.

Note

If you store both events and a current-state attribute, you must decide which one is authoritative. If both can disagree, the model is not well-defined.

11.5 Status as a state machine

Many domains have "status" attributes: appointment status, reservation status, loan status. Status modelling is often handled poorly because it is treated as a free-form string.

At minimum, status should come from a defined set:

$$status \in \{\text{scheduled, cancelled}\}$$

That is a domain constraint.

For more complex domains, statuses behave like a state machine. Some transitions are allowed; others are not. A reservation might transition from `active` to `fulfilled` or `cancelled`, but not back to `active` without a new action. If your book intends to teach correctness, it is useful to express this as rules.

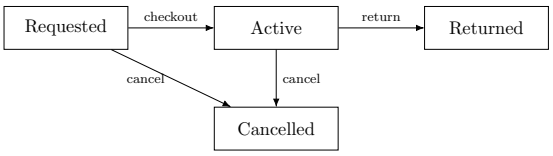


Figure 11.1: A status lifecycle can be modeled as explicit transitions.

Example

Status transition rules (reservation)
R1: active → fulfilled is allowed.
R2: active → cancelled is allowed.
R3: fulfilled → active is not allowed.

Implementation details can wait. The modelling point is that status is not only a value; it is behaviour.

11.5.1 Diagram 1: reservation lifecycle (described)

Imagine a simple state diagram with three nodes: `active`, `fulfilled`, `cancelled`. Arrows go from `active` to the other two states. There are no arrows back to `active`.

11.5.2 Diagram 2: loan lifecycle (described)

Imagine a diagram with nodes `checked_out`, `returned`, `lost`. Arrows go from `checked_out` to `returned` and `lost`. A `lost` item may later be marked `returned`, but that transition should be explicit in rules.

11.6 Temporal constraints patterns

The hardest constraints in introductory relational design are temporal. Two common patterns appear repeatedly.

11.6.1 Pattern 1: No overlap (interval conflicts)

Clinic scheduling is a textbook example. Each appointment defines an interval:

$$[start\_time,\ start\_time + duration)$$

The constraint "a doctor cannot have overlapping scheduled appointments" means: for any doctor, no two scheduled appointment intervals may overlap.

This constraint cannot be captured by a simple uniqueness constraint when durations vary. You can still specify it clearly as a business constraint.

Example

No-overlap constraint (conceptual)
For any doctor, for any two appointments A and B with status = scheduled,
the intervals of A and B must not overlap.

The schema must store enough information to evaluate overlap. That means start time and a duration (or end time). If you only store a start time with variable duration hidden elsewhere, you cannot validate the constraint.

11.6.2 Pattern 2: One active record per entity

In the library, the core rule is: a copy has at most one active loan at a time.

This is a temporal constraint expressed via state. If "active" is defined as "return_date is missing", then the rule becomes: for each copy, there may be at most one loan with return_date missing. This is a constraint over a filtered subset of rows.

Example

One-active constraint (conceptual)
For each copy, there must be at most one loan such that return_date is missing.

The modelling lesson is that "active" must be defined precisely. If "active" is not defined, correctness cannot be enforced or tested.

11.6.3 Pattern 3: Effective dating

Effective dating stores validity periods explicitly:

$$valid\_from, valid\_to$$

The rule is that periods must not overlap for the same entity when only one value should be active. This pattern is common for prices, memberships, and roles.

11.7 History: overwrite versus append

When values change over time, you have two general approaches.

Overwrite means you store only the current value. This is simpler but loses history.

Append means you store changes as new rows (events or history records). This preserves history but increases structure.

Both can be correct depending on the domain. For teaching, it helps to show a common pattern: a current-value table plus an optional history table.

Example

Optional history pattern

Current state:

$$\textbf{Member}(\underline{member\_id}, name, email)$$

History:

$$\textbf{MemberEmailHistory}(\underline{member\_id} \rightarrow \textbf{Member}, \underline{changed\_at}, old\_email, new\_email)$$

This preserves important changes without forcing every attribute into a history structure.

11.8 Common mistakes

Time modelling mistakes are common and costly.

Mistake: Storing only current state when history is required.
Fix: Add event or history relations for changes that matter.

Mistake: Defining “active” informally.
Fix: Write a precise definition tied to attributes and time.

Mistake: Mixing end times and durations inconsistently.
Fix: Choose one representation and derive the other if needed.

Mistake: Allowing overlapping validity periods unintentionally.
Fix: Add no-overlap constraints for effective-dated records.

11.9 Mini checklist

Before you finalize time modelling, check:

- Which facts are events and which are state?
- Which source of truth is authoritative (events or state)?
- How is “active” defined for each temporal rule?
- Are overlap rules explicit for intervals or validity periods?
- Is the time representation consistent across relations?
- Do you need history, and if so, where is it stored?
- Are status transitions specified as rules?

Summary

Time forces clarity. You must distinguish entities from events, and decide whether state is stored, derived, or both. Status attributes should be constrained and, in richer domains, treated as state machines with explicit allowed transitions. Temporal constraints such as "no overlap" and "one active at a time" must be defined precisely, because correctness depends on those definitions.

Exercises

Answer in full sentences.

1. In a clinic model, define precisely what it means for two appointments to overlap.
2. In a library model, define precisely what "active loan" means and propose two alternative definitions.
3. Choose one entity and decide which of its attributes should keep history. Justify why.
4. Write three status transition rules for a concept in your chosen domain.
5. Identify one place where storing both events and current state might help, and state which one is authoritative.
6. Write a validity-period rule using valid_from and valid_to for a membership.
7. Propose a no-overlap constraint for room bookings and define the overlap test.
8. Give an example of a temporal rule that cannot be fully enforced and explain how to detect violations.

In the next chapter we examine advanced relationship modelling. The time and state patterns here will reappear in specializations and optionality decisions.

Chapter 12

Relationships in Practice: Optionality, Specialization, and Association

Learning objectives

In this chapter you will deepen your understanding of relationships. You will learn how optionality affects schema structure, how to model specialization (subtypes) without ambiguity, and how to treat association relations as first-class parts of the model when they carry meaning.

12.1 Optionality is a design decision

Optionality is often treated as a small detail: "nullable or not". In reality, optionality is a statement about what the domain allows. It affects correctness, lifecycle, and how you validate data.

Consider the sentence: "An appointment has a doctor." This implies mandatory participation. But consider: "A patient may have an insurance profile." This implies optional participation. The difference is not about storage. It is about whether the system accepts the state "patient exists but no insurance profile exists".

Optionality decisions should be explicit in the rules. If optionality is left implicit, the schema will often drift into permissiveness, allowing states that the domain does not allow.

Example

Optionality in words
Mandatory: "Each appointment is with exactly one doctor."
Optional: "A patient may have an insurance profile."

12.1.1 Optionality and foreign key placement

In one-to-one relationships, optionality guides where the foreign key goes. If one side can exist without the other, place the foreign key on the optional side and enforce uniqueness there. This reduces nulls and matches typical creation workflows.

In one-to-many relationships, optionality decides whether the foreign key can be missing. For example, if an appointment must reference a doctor, `doctor_id` is mandatory. If you make it optional, you allow "unassigned appointments", which might be correct in some domains and incorrect in others. The schema should follow the rule, not the other way around.

12.2 Association relations are not “just join tables”

Many-to-many relationships are represented with association relations. Beginners often treat these as purely technical structures, but they are often conceptually central. An association relation is not a workaround. It is the correct representation of a relationship that has its own identity.

When an association has attributes, it becomes clearly meaningful. In a university domain, enrollment is the association between student and course, and it has attributes such as enrollment date and grade. In a clinic domain, doctor specialties may include the date the doctor was certified.

Even when an association has no attributes, it still represents meaning: membership, eligibility, assignment, coverage.

Definition

An *association relation* represents a many-to-many relationship. It becomes conceptually central when the relationship carries attributes or when it represents an event or membership over time.

Example

Enrollment as a first-class association

$$\textbf{Enrollment}(\underline{student\_id} \to \textbf{Student},\ \underline{course\_id} \to \textbf{Course}, enrolled\_at, grade?)$$

Enrollment is not a technical join; it is a meaningful entity in the domain.

A good teaching move is to ask: “If I delete this association row, what real-world thing disappears?” If the answer is “an enrollment” or “a certification” or “a membership”, then you are looking at a real concept, not a technical artefact.

12.3 Specialization: modelling subtypes

Many domains contain “kinds of” entities. A user can be a student or a staff member. An appointment can be an exam or a consultation. A payment can be a card payment or a bank transfer.

This is called *specialization* or *subtyping*. The key modelling question is: do the subtypes have distinct attributes or constraints that matter? If not, they can remain as a simple attribute. If yes, subtyping becomes useful.

Definition

A *specialization* models a supertype entity with one or more subtypes. Subtypes inherit identity from the supertype and add subtype-specific attributes or constraints.

12.3.1 Two constraints: disjointness and completeness

Subtype models have two structural rules.

Disjointness answers: can an instance belong to more than one subtype? A person might be both a staff member and a student, depending on the domain. In other domains, a vehicle is either a car or a motorcycle, but not both.

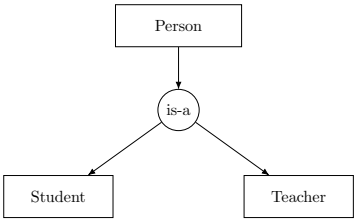


Figure 12.1: A specialization shows a supertype with subtypes linked by an is-a relationship.

Completeness answers: must every supertype instance belong to at least one subtype? Sometimes yes (every payment is exactly one payment type). Sometimes no (you may create a person record before deciding their role).

These rules must be stated. If they are not, the subtype model becomes ambiguous.

Example

Subtype constraints in words
Disjoint: “An appointment is either an exam or a consultation, not both.”
Overlapping: “A person may be both a student and a staff member.”
Complete: “Every payment must have exactly one payment subtype.”
Incomplete: “A person may exist without a role assigned yet.”

12.4 Implementing subtypes in a relational schema (concept-first)

There are three common relational patterns for subtypes. We describe them conceptually without committing to SQL.

12.4.1 Pattern 1: Single table with a type attribute

You store all attributes in one relation and add a type attribute. This works when subtype-specific attributes are few, optional, and not essential for correctness.

Example:

$$\textbf{Appointment}(\underline{appointment\_id}, \ldots, \\ type \in \{\text{exam, consultation}\}, exam\_room?, exam\_code?)$$

The risk is that you end up with many nullable columns and weak enforcement of subtype constraints.

12.4.2 Pattern 2: Supertype + subtype tables (shared primary key)

You create one relation for the supertype and one for each subtype. Subtype relations use the same primary key value as the supertype. This is clean when subtype attributes matter and you want strong structure.

Example

Supertype + subtype schema

$$\textbf{Payment}(\underline{payment\_id}, amount, created\_at)$$
$$\textbf{CardPayment}(\underline{payment\_id} \rightarrow \textbf{Payment}, last4, auth\_code)$$
$$\textbf{BankTransfer}(\underline{payment\_id} \rightarrow \textbf{Payment}, iban, reference)$$

This pattern represents inheritance clearly. It also matches the conceptual idea: every card payment is a payment.

12.4.3 Pattern 3: Separate tables per subtype (no explicit supertype)

Sometimes you keep completely separate relations for each subtype. This can work when subtypes are operationally separate and rarely treated uniformly. But it becomes awkward if you need cross-subtype reporting or shared relationships. For teaching, this pattern is often less helpful as a default.

12.5 Subtype modelling decision guide

Deciding whether to use subtypes should be explicit. Use the following criteria:

- subtype-specific attributes are substantial,
- subtype-specific constraints are important,
- subtype lifecycles differ (states or transitions),
- completeness/disjointness must be enforced,
- reports or rules treat subtypes differently.

12.5.1 Attribute versus subtype

If type is mostly for labeling or filtering, and there are no meaningful subtype-specific constraints, an attribute is often enough. If correctness depends on subtype-specific rules or attributes, use subtypes.

12.5.2 Optionality and completeness

If subtype membership is incomplete, you allow a supertype instance with no subtype row. If it is complete, you must ensure every supertype row has exactly one subtype row (disjoint) or at least one subtype row (overlapping). Subtype structures are only correct when their disjointness and completeness rules are specified and validated.

12.6 Worked example: Payment subtypes

Assume payments can be made by card or bank transfer. Each subtype has attributes that matter for correctness.

Example

Payment subtypes (complete, disjoint)

$$\textbf{Payment}(\underline{payment\_id}, amount, created\_at)$$

$$\textbf{CardPayment}(\underline{payment\_id} \rightarrow \textbf{Payment}, last4, auth\_code)$$

$$\textbf{BankTransfer}(\underline{payment\_id} \rightarrow \textbf{Payment}, iban, reference)$$

Every payment is exactly one of these subtypes.

12.7 Completeness and disjointness test cases

Subtype rules should be validated with simple tests.

Test 1: Can a **Payment** exist without any subtype row? If not, completeness is required.
Test 2: Can a **Payment** be both card and bank transfer? If not, disjointness is required.
Test 3: Are subtype-specific attributes allowed to be missing? If not, subtype tables are needed.

12.8 Anti-patterns

Subtype modelling has common traps.

EAV misuse: modelling subtypes as generic attribute/value rows hides constraints.
Type field with many nulls: a single table with dozens of optional columns weakens correctness.

12.9 Common mistakes

Mistakes here often lead to inconsistent data.

Mistake: Using a type attribute without enforcing subtype rules.
Fix: Add subtype tables or explicit constraints.

Mistake: Placing shared relationships on subtype tables.
Fix: Keep shared relationships on the supertype.

Mistake: Leaving disjointness/completeness unspecified.
Fix: State and validate these rules explicitly.

12.10 Mini checklist

Before finalizing subtypes, check:

- Are subtype-specific attributes meaningful and required?
- Is completeness specified (every supertype has a subtype)?
- Is disjointness specified (can instances be in multiple subtypes)?
- Are shared relationships placed on the supertype?
- Are subtype rules validated with test cases?
- Would a simple type attribute be sufficient instead?
- Are anti-patterns (EAV, many nulls) avoided?

Summary

Optionality is a statement about allowed states, not a nullable column choice. Association relations represent many-to-many meaning and often become central entities when they carry attributes or time semantics. Specialization (subtypes) is useful when "kinds of" entities have distinct attributes, constraints, or lifecycles. Subtype models require explicit rules about disjointness and completeness, and the relational pattern should be chosen to preserve those rules clearly.

Exercises

Answer in full sentences and then write the corresponding schema fragments in book notation.

1. Give one example of a relationship where optionality is mandatory in your domain and justify why.
2. Create a many-to-many relationship and design an association relation with at least two relationship attributes.
3. Choose a concept (payment, appointment, user role) and decide whether to model it as a subtype or a type attribute. Justify your choice.
4. If you chose subtypes, state whether they are disjoint and whether the specialization is complete.
5. Design a supertype + subtype schema fragment using shared primary keys.
6. Write two test cases that would detect violations of completeness or disjointness.
7. Show an anti-pattern using a type field with many nulls and explain how you would fix it.
8. Add subtype-specific constraints to your schema and explain where they belong.

In the next chapter we formalize dependencies. Subtypes and associations often reveal hidden functional dependencies.

Chapter 13

Functional Dependencies as a Design Tool

Learning objectives

In this chapter you will learn to use functional dependencies to justify schema structure. You will be able to write dependencies from domain meaning, detect when a relation mixes facts that should be separated, and explain normalization decisions as correctness decisions rather than as formalisms.

13.1 Why dependencies matter

When people first learn normalization, it often feels like a set of abstract rules. In practice, normalization is a consequence of a simple idea: some facts determine others. If you place facts together in a relation where the key does not determine them, you create redundancy. That redundancy later becomes anomalies.

Functional dependencies are a language for stating "what determines what" precisely. They are not only a theoretical tool. They are a way to make your design decisions auditable.

Definition

A *functional dependency* $X \to Y$ means: if two rows agree on X, they must also agree on Y. In plain language: knowing X uniquely determines Y.

You do not need heavy notation to benefit from this. You need the habit of writing a few dependencies and checking whether your schema respects them.

13.2 How to write dependencies from rules

A common mistake is to infer dependencies from a small dataset. Dependencies should come from domain rules, policies, and definitions. They are claims about reality as modelled by the system.

13.2.1 Example 1: identity-based dependency

If `member_id` is the identity of a member, then:

$$member\_id \to name,\ email$$

This is a statement about identity, not about observed data.

13.2.2 Example 2: catalog dependency

If ISBN identifies a title in your domain, then:

$$isbn \to title,\ author$$

If ISBN is optional or unreliable, this dependency may not hold, and the key choice should change.

13.3 The dependency test for mixed relations

Suppose you have a relation with key K and attributes A, B, C. If the meaning implies that some attribute B is determined by something other than K, you are probably mixing facts.

This is the heart of normalization, stated as a practical test.

Principle

Dependency test
In a well-structured relation, non-key attributes should be determined by the key. If a non-key attribute is determined by something else, you likely need a new relation.

This is not a rigid law. It is a strong signal. When you break it deliberately, you should document why.

13.4 Worked example: clinic scheduling

Consider a naive relation:

$$\textbf{AppointmentRecord}(\underline{appointment\_id}, start\_time, doctor\_name, doctor\_specialty, patient\_name)$$

If `appointment_id` is the key, this relation claims:

$$appointment\_id \to doctor\_name,\ doctor\_specialty,\ patient\_name$$

But from meaning, we also have:

$$doctor\_id \to doctor\_name,\ doctor\_specialty$$
$$patient\_id \to patient\_name$$

So doctor facts and patient facts are being repeated across many appointments. That repetition is what creates update anomalies.

The correct structural move is to separate entities:

$$\textbf{Doctor}(\underline{doctor\_id}, name, specialty)$$

$$\textbf{Patient}(\underline{patient\_id}, name)$$

$$\textbf{Appointment}(\underline{appointment\_id}, start\_time, doctor\_id \to \textbf{Doctor}, patient\_id \to \textbf{Patient})$$

Now the dependencies align with keys:

$$doctor\_id \to name,\ specialty$$
$$patient\_id \to name$$
$$appointment\_id \to start\_time,\ doctor\_id,\ patient\_id$$

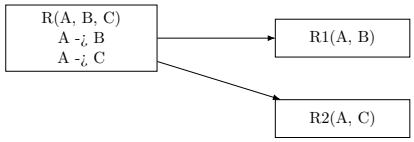


Figure 13.1: Dependencies guide decomposition into relations whose keys determine their attributes.

13.5 Worked example: address modelling

Suppose you store a customer's address directly in **Customer**:

$$\textbf{Customer}(\underline{customer\_id}, name, street, city, postal\_code)$$

If customers can have multiple addresses, this structure is wrong. The dependency is:

$$customer\_id \twoheadrightarrow street,\ city,\ postal\_code$$

because a customer can have more than one address.

The correction is to introduce an association:

$$\textbf{Address}(\underline{address\_id}, street, city, postal\_code)$$

$$\begin{aligned}\textbf{CustomerAddress}(&\underline{customer\_id} \to \textbf{Customer},\\ &\underline{address\_id} \to \textbf{Address},\\ &address\_type)\end{aligned}$$

Now address facts depend on `address_id`, and the association captures the relationship.

13.6 Transitive dependencies and why they create redundancy

A transitive dependency happens when:

$$K \to A \quad \text{and} \quad A \to B$$

so $K \to B$ indirectly.

In design terms, this usually means you stored a code and its description in the same relation. For example, storing `specialty_name` inside **Doctor** when `specialty_id` exists and determines the name.

The redundancy is subtle at first, then painful later: multiple spellings, inconsistent naming, and unclear governance.

The structural fix is to create a separate relation:

$$\textbf{Specialty}(\underline{specialty\_id}, name)$$

and reference it from **Doctor**.

13.7 Dependencies and many-to-many relationships

Dependencies also explain why many-to-many relationships require association relations.

If a doctor can have many specialties and a specialty can belong to many doctors, then neither of these holds:

$$doctor\_id \to specialty\_id \quad \text{nor} \quad specialty\_id \to doctor\_id$$

So the relationship itself must be represented as its own structure:

$$\textbf{DoctorSpecialty}(\underline{doctor\_id} \to \textbf{Doctor}, \underline{specialty\_id} \to \textbf{Specialty})$$

This is not an implementation trick. It is the correct representation of meaning.

13.8 Dependencies as review language

Dependencies are not just for normalization. They give reviewers a precise language for critique.

When you say "this relation mixes facts," you can support it with a dependency:

$$A \to B \quad \text{but} \quad A \text{ is not a key}$$

This turns a vague concern into a concrete design signal.

Principle

Dependency review checklist
For a relation $\mathbf{R}(\underline{K}, A_1, \ldots, A_n)$:

- Write 2–5 dependencies that come from meaning.
- Check that non-key attributes are determined by K.
- If you find $A \to B$ where A is not a key, ask whether you mixed facts.

Summary

Functional dependencies express what determines what. They come from domain meaning, not from sample data. They provide a practical test for mixed relations: non-key attributes should be determined by the key. When a non-key attribute is determined by something else, you are likely storing redundant facts and inviting anomalies.

Exercises

Answer in full sentences and then write dependencies.

1. Choose a domain and write a relation that you suspect is mixing facts.
2. Propose a primary key and write the implied dependency $K \to$ (all other attributes).
3. Write two additional dependencies that come from meaning and show why redundancy exists.
4. Restructure the schema into at least two relations and rewrite the dependencies so they align with keys.
5. Give one example of a transitive dependency and show how you would eliminate it structurally.

Chapter 14

Normalization Patterns in Real Domains

Learning objectives

In this chapter you will learn a set of recurring normalization patterns used in real relational designs. You will be able to recognise the underlying dependency behind each pattern, choose an appropriate schema structure, and explain what anomaly the pattern prevents.

14.1 Why patterns help

After you learn the workflow and the core mapping rules, most design work becomes recognition. You see a requirement and you match it to a known structure. Patterns are not shortcuts for thinking. They are a way to avoid repeating the same mistakes.

Each pattern below is presented with:

- the situation in plain language,
- the dependency intuition,
- a compact schema in book notation,
- and the anomaly it prevents.

14.2 Pattern 1: Order and OrderLine (the line-item pattern)

14.2.1 Situation

A customer places an order. An order contains multiple items. Each item has a quantity and a price at the time of purchase.

The modelling trap is to store "items" as repeating groups inside **Order**, or to store order attributes inside the line table.

14.2.2 Dependency intuition

Order facts are determined by `order_id`. Line facts are determined by (order_id, line_no) or by a surrogate `order_line_id`. Product facts are determined by `product_id`. Do not mix them.

14.2.3 Schema

$$\textbf{Customer}(\underline{customer\_id}, name, email \text{ [unique]})$$

$$\textbf{Order}(\underline{order\_id}, created\_at, status, customer\_id \rightarrow \textbf{Customer})$$

$$\textbf{Product}(\underline{product\_id}, sku \text{ [unique]}, name, list\_price)$$

$$\textbf{OrderLine}(\underline{order\_id} \rightarrow \textbf{Order}, \underline{line\_no}, product\_id \rightarrow \textbf{Product}, quantity, unit\_price)$$

14.2.4 What it prevents

This structure prevents update anomalies like "customer name duplicated across order lines". It also supports history correctly: `unit_price` is stored at purchase time, independent of future `list_price` changes.

14.3 Pattern 2: Address modelling (entity vs attribute)

14.3.1 Situation

A system stores people or organisations and their addresses. People can have multiple addresses (billing, shipping, current, previous). Addresses can change over time.

A common failure is to place address fields directly on **Customer**: `street1, street2, city, zip, ....` That fails immediately when there is more than one address.

14.3.2 Dependency intuition

A customer does not determine a single address. Instead, a customer determines a set of address relationships, possibly with types and validity periods.

14.3.3 Schema (address as entity)

$$\textbf{Address}(\underline{address\_id}, street, city, postal\_code, country)$$

$$\textbf{CustomerAddress}(\underline{customer\_id} \rightarrow \textbf{Customer}, \underline{address\_id} \rightarrow \textbf{Address}, type, valid\_from, valid\_to?)$$

14.3.4 What it prevents

This prevents repeating groups and allows history. It also avoids mixing "what the address is" with "how the customer uses it" (billing/shipping), which belongs to the association.

14.4 Pattern 3: Code and description (the lookup pattern)

14.4.1 Situation

You store a code (status, category, specialty, role) and also store its human-readable name in multiple places.

This often creates inconsistent spellings and conflicting definitions.

14.4.2 Dependency intuition

If `status_code` determines `status_name`, then storing both together repeatedly creates redundancy. The name belongs in one place.

14.4.3 Schema

$$\textbf{Status}(\underline{status\_code}, name, description)$$

$$\textbf{Appointment}(\underline{appointment\_id}, start\_time, status\_code \rightarrow \textbf{Status}, \dots)$$

14.4.4 What it prevents

This prevents update anomalies where "cancelled" appears as "canceled" in some rows. It also makes governance explicit: status definitions live in one relation.

14.5 Pattern 4: Enrollment (association with attributes)

14.5.1 Situation

A student can enroll in many courses. A course has many students. Enrollment has attributes like enrollment date, grade, and status.

14.5.2 Dependency intuition

Neither `student_id` nor `course_id` determines the other. The association is the identity. Enrollment attributes are determined by the pair.

14.5.3 Schema

$$\textbf{Student}(\underline{student\_id}, name, email)$$

$$\textbf{Course}(\underline{course\_id}, code\ [\text{unique}], title)$$

$$\textbf{Enrollment}(\underline{student\_id} \rightarrow \textbf{Student}, \underline{course\_id} \rightarrow \textbf{Course}, enrolled\_at, status, grade?)$$

14.5.4 What it prevents

This prevents forced duplication such as storing a list of student ids inside a course row. It also keeps enrollment history and supports constraints like "a student can be enrolled at most once in a course per term" (if you add `term`).

14.6 Pattern 5: Inventory by location (context-dependent identity)

14.6.1 Situation

A product exists across multiple locations (stores, warehouses). You want quantity-on-hand per location.

A common mistake is to store one `quantity` on **Product**. That loses the location dimension.

14.6.2 Dependency intuition

Quantity is not determined by product alone. It is determined by (product, location).

14.6.3 Schema

$$\textbf{Location}(\underline{location\_id}, name)$$

$$\textbf{StockLevel}(\underline{product\_id} \rightarrow \textbf{Product}, \underline{location\_id} \rightarrow \textbf{Location}, quantity\_on\_hand)$$

14.6.4 What it prevents

This prevents inconsistent inventory reporting and forces the model to represent location as a first-class dimension.

14.7 Pattern 6: Audit log (append-only event history)

14.7.1 Situation

You need a history of changes: who changed what, when, and from which value to which value.

A naive approach is to overwrite values and keep no record. Another naive approach is to add many "previous_*" columns.

14.7.2 Dependency intuition

History is not an attribute of the entity. It is a stream of events, each with its own time and author.

14.7.3 Schema

$$\textbf{AuditEvent}(\underline{event\_id}, entity\_type, entity\_id, action, actor, occurred\_at)$$

$$\textbf{AuditFieldChange}(\underline{event\_id} \rightarrow \textbf{AuditEvent}, \underline{field\_name}, old\_value, new\_value)$$

14.7.4 What it prevents

This avoids lossy overwrites and keeps traceability. It also keeps audit structure separate from domain entities, preventing clutter and anomalies in core tables.

14.8 How to apply patterns without forcing them

Patterns are helpful, but they can be applied blindly. The correct approach is:

- start from domain meaning and rules,
- write a few dependencies in plain language,
- then choose the pattern that makes those dependencies align with keys.

When you choose a pattern, you should be able to say: *"This prevents this specific anomaly."* If you cannot, you may be restructuring for style rather than correctness.

Summary

Most relational schemas are combinations of a small set of patterns. Line items separate order facts from item facts. Addresses become entities because customers can have many addresses and address usage is a relationship. Lookup tables centralize code meanings. Association relations represent many-to-many membership and carry relationship attributes. Context-dependent facts (like stock per location) belong to the pair. Audit logs are append-only event history, separate from core entities.

Exercises

Pick one domain you know and answer in full sentences, then write the schema fragments.

1. Identify one place where you need a line-item pattern and write **Order**/**OrderLine**-

style relations.

2. Identify one attribute that is really a lookup code and design the lookup relation.
3. Identify one many-to-many relationship that needs an association relation with attributes.
4. Identify one context-dependent fact (product-by-location, member-by-branch, etc.) and model it.
5. Choose one entity and propose a minimal audit logging structure for changes to it.

Chapter 15

Design Trade-offs: Correctness, Complexity, and Change

Learning objectives

In this chapter you will learn how to reason about trade-offs in relational design. You will be able to justify when to keep a design strict versus flexible, when to introduce new relations versus keep attributes, and how to plan for change without making the schema vague.

15.1 Why trade-offs belong in a design book

If a database design book only teaches "rules", it teaches a false picture. Real design is constrained by change, deadlines, governance, and the fact that domains evolve.

Trade-offs are not excuses for weak design. They are decisions that should be explicit and documented. A good schema is not the one with the most tables. It is the one that preserves meaning while staying maintainable under expected change.

Definition

A *design trade-off* is a deliberate choice between competing goals (e.g., strictness vs flexibility) where no single option is universally correct. A trade-off is acceptable only if the consequences are understood and controlled.

15.2 Trade-off 1: Strict schema versus flexible schema

Some teams try to represent everything as structured relations with tight constraints. Others push variability into "free-form" fields such as JSON blobs or generic key-value tables.

Strict structure improves correctness and queryability. Flexibility improves speed of iteration and handles unknown future attributes. The danger is that flexibility often becomes ambiguity.

A useful rule is to be strict about:

- identity,
- relationships that matter for correctness,
- and constraints that prevent invalid states.

Be flexible mainly about:

- optional descriptive attributes that do not affect core rules,
- rapidly changing metadata where enforcement is not critical.

15.3 Trade-off 2: Attribute versus entity

A recurring modelling question is whether something deserves its own entity. For example: Specialty, Category, Department, PaymentMethod.

You can model "specialty" as a string attribute on **Doctor**. Or you can model it as a **Specialty** entity referenced by **Doctor**. Neither is always correct. The decision depends on meaning and governance.

15.3.1 When an attribute is enough

An attribute is often enough when:
- the value is purely descriptive,
- it is not governed as a controlled list,
- it does not carry additional properties,
- and it does not participate in relationships.

15.3.2 When you need an entity

An entity is usually the right choice when:
- the concept has its own identity and lifecycle,
- you need a controlled list with stable meanings,
- it carries additional attributes (description, category, validity),
- or it participates in relationships (many-to-many).

The safe way to teach this is to frame it as a dependency question: if you have `specialty_id` and `specialty_name`, and $specialty\_id \rightarrow specialty\_name$, then an entity is often appropriate.

15.4 Trade-off 3: Surrogate keys versus natural keys

This book already introduced the key decision. Here we add a decision-making frame.

Natural keys reduce joins and align with domain meaning. But they can be missing at creation time, can change, or can be governed externally. Surrogate keys are stable, but require explicit uniqueness constraints on natural identifiers if those identifiers matter.

A practical compromise is common: use surrogate primary keys for stability, and enforce natural uniqueness separately.

The trade-off is acceptable only if you document what the natural uniqueness constraints are and how changes are handled.

15.5 Trade-off 4: Normalization versus denormalization

Normalization reduces anomalies and makes correctness easier. Denormalization can improve performance or simplify read models, but it reintroduces redundancy.

The key is not to forbid denormalization. The key is to make it explicit:
- what is duplicated,
- what anomaly it could create,
- and what mechanism keeps it consistent.

In many systems, denormalization is not a schema choice but a system architecture choice. You keep a normalized source of truth, and you build denormalized read models for queries. Even without discussing specific tools, the design lesson remains: keep one authoritative representation.

15.6 Trade-off 5: Derived values versus stored values

Many values can be derived from other data. Examples include “is overdue”, “active loan count”, “account balance”, “available copies”.

Storing derived values improves speed but risks inconsistency. Deriving values improves correctness but can be expensive.

A safe guideline: store derived values only when:

- they are expensive to compute frequently,
- you can define exactly when and how they update,
- and you can recover them from the source data if needed.

If you cannot explain the update rule, you should not store the derived value.

15.7 Trade-off 6: Hard deletion versus soft deletion

Deletion is a correctness decision. If you delete a patient, what happens to their appointments? If you delete a member, what happens to their loan history?

Hard deletion removes history and can break references. Soft deletion preserves history but adds state complexity.

A reasonable default for many domains is: do not hard-delete core entities that participate in historical events. Instead, use an “inactive” state or an archive mechanism. This is not a database feature; it is a modelling decision about preserving meaning over time.

15.8 Making trade-offs auditable

Trade-offs become dangerous when they are implicit. The simplest fix is a short decision record for each major trade-off. The goal is not paperwork. The goal is that a future reader can understand why the model looks like this.

Example

Decision record (template)

Decision: (one sentence).
Alternatives: (two or three).
Rationale: (why this is chosen, in domain terms).
Consequences: (what you must now enforce, monitor, or accept).

15.9 A short framework: expected change

A useful way to decide is to think about which parts of the domain are stable and which will change.

Ask:

- Which identifiers will never change?
- Which relationships are core and must be enforced?
- Which attributes will evolve often (metadata)?
- Which constraints are policy-driven and likely to change?

Then you design strictness around the stable core. You design flexibility around the evolving edges. This keeps the schema meaningful without making it brittle.

Summary

Trade-offs are unavoidable in relational design, but they must be explicit. Strictness protects identity, relationships, and constraints. Flexibility should be limited to non-core descriptive information. Key choices, normalization boundaries, derived values, and deletion policies must be justified and documented. A good design is not the most complex; it is the clearest under expected change.

Exercises

Pick one domain (clinic, library, university, online shop) and answer in full sentences.

1. Identify one place where you would choose an attribute over an entity and justify it.
2. Identify one place where you would choose an entity over an attribute and justify it.
3. Choose one derived value and decide whether to store it or derive it. State the update rule if stored.
4. Decide whether you allow hard deletion for one core entity. Justify your policy.
5. Write two decision records using the template above for two trade-offs in your model.

Chapter 16

A Design Review Method: How to Critique a Schema

Learning objectives

In this chapter you will learn a structured method to review and critique a relational design. You will be able to conduct a design review using a small set of questions, identify common failure modes, and produce actionable feedback backed by rules, constraints, and dependencies.

16.1 Why a review method matters

Most schema problems are not caused by ignorance of the relational model. They are caused by missing review. A design can look plausible while still allowing invalid states or forcing future duplication.

A review method is valuable because it makes critique repeatable. It also makes feedback objective: instead of "I don't like this table", you can say "this relation mixes facts that imply the dependency $A \to B$ but A is not a key".

Definition

A *design review* is a structured evaluation of a schema and its documentation to check correctness, clarity, and maintainability. A good review produces specific issues and specific actions.

16.2 Inputs to a review

A review cannot be done from a schema alone. You need a minimal context set.

A good review package includes:

- a short domain narrative (6–12 sentences),
- a numbered business rule list,
- the logical schema in book notation,
- a constraint block (C1, C2, ...),
- 6–10 key questions the database must support.

If one of these is missing, the review should start by requesting it, because otherwise critique becomes guessing.

16.3 The review flow

Use the same flow every time. It keeps discussions focused and prevents people from jumping to performance too early.

Principle

Design review flow

1. Domain clarity: narrative and vocabulary.
2. Identity: keys and uniqueness.
3. Relationships: cardinality, optionality, and association relations.
4. Constraints: what is enforced, what is missing.
5. Redundancy: dependencies and normalization signals.
6. Time and state: events, history, and temporal constraints.
7. Validation: key questions and paper test data.

Each step has a small set of concrete checks.

16.4 Step 1: Domain clarity checks

A schema cannot be correct if the domain is not precise.

Check:

- Are core terms defined (e.g., title vs copy, appointment vs visit)?
- Are there overloaded words (e.g., “user”, “status”, “date”) used without definition?
- Are the main business rules written as testable statements?

Common issue: the narrative mixes two concepts under one name. Fix: split vocabulary first, then redesign.

16.5 Step 2: Identity checks

Identity failures are catastrophic because they break references.

Check:

- Does every entity have a primary key?
- Is the key stable (does it change)? Is it available at creation time?
- Are natural identifiers that must be unique actually constrained as unique?
- Are there any “keys” that are actually descriptions (e.g., name as primary key)?

Common issue: using mutable business data as a primary key. Fix: introduce surrogate identity and constrain the natural identifier separately.

16.6 Step 3: Relationship checks

Relationship structure often reveals conceptual mistakes.

Check:

- Are 1:N relationships represented with FKs on the N side?
- Are N:M relationships represented with association relations?
- Are relationship attributes stored on the association/event relation (not on entities)?

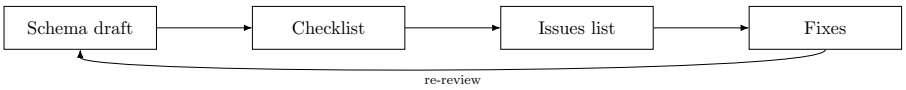


Figure 16.1: A repeatable review flow turns critique into a controlled loop.

- Is optionality stated and consistent with the rules?

Common issue: encoding many-to-many as a comma-separated list. Fix: create an association relation.

16.7 Step 4: Constraint checks

Constraints make the model safe.

Check:

- Are entity integrity and referential integrity constraints stated?
- Are domain constraints stated (allowed sets, non-negative values, date ordering)?
- Are the top 3–5 business constraints stated explicitly?
- For each business constraint: is enforcement feasible, or is detection documented?

Common issue: the schema implies constraints in prose, but there is no constraint block. Fix: create a numbered constraint block and map rules to constraints.

16.8 Step 5: Redundancy and dependency checks

Redundancy is often hidden. Dependencies expose it.

Check:

- For each relation, can you state what determines each attribute?
- Do any non-key attributes determine other non-key attributes (transitive dependencies)?
- Are code/name pairs duplicated (status_code and status_name stored together repeatedly)?
- Are entity attributes stored inside association relations (2NF violations)?

Common issue: a relation mixes entity facts and event facts. Fix: split into entity relations plus event/association relations.

16.9 Step 6: Time and state checks

Time-heavy domains break naive schemas.

Check:

- Are events modelled as entities (Loan, Appointment, Payment)?
- Is "active" defined precisely where it matters (active loan, active reservation)?
- Are temporal constraints stated (no overlap, one active at a time)?
- Is history preserved where needed, or is overwrite acceptable and documented?

Common issue: storing current state on an entity without event history. Fix: introduce an event relation and decide authoritative source of truth.

16.10 Step 7: Validation checks

Validation makes critique concrete.

Check:

- Can the schema support the key questions without duplication?
- Can you represent two valid scenarios with small paper test data?
- Can you represent an invalid scenario as a clear constraint violation?
- Is there a rule-to-schema traceability map for the important rules?

Common issue: the schema supports the "happy path" but not exceptions (cancellations, returns, replacements). Fix: add state/time modelling and constraints.

16.11 How to write actionable review comments

Good review comments do two things: they identify the issue, and they propose a specific repair with justification.

A practical format:

- **Observation:** what is wrong, referencing a relation/attribute.
- **Risk:** what anomaly or invalid state it allows.
- **Fix:** what structural change to make.
- **Justification:** which rule/dependency/constraint motivates it.

Example

Example review comment

Observation: **AppointmentRecord** stores doctor_name and doctor_specialty per appointment.
Risk: update anomaly when a doctor changes specialty; inconsistent doctor naming.
Fix: introduce **Doctor** and reference it from **Appointment**.
Justification: doctor_id → name, specialty; doctor facts should be determined by doctor identity, not by appointment.

This tone stays technical and avoids subjective critique.

16.12 A short review checklist (printable)

Use this as a one-page checklist when reviewing schemas.

Example

Checklist

- Domain: terms defined; rules testable.
- Keys: every entity has a PK; uniqueness constraints stated.
- Relationships: correct 1:N mapping; N:M via association relation; optionality explicit.
- Constraints: entity/ref integrity; domain checks; top business constraints.
- Redundancy: no mixed facts; dependencies align with keys; no transitive duplication.
- Time: events as entities; “active” defined; temporal constraints stated.
- Validation: key questions; traceability; paper test scenarios.

Summary

A design review should be repeatable. Start from domain clarity, then check identity, relationships, constraints, redundancy, time/state, and validation. Use dependencies to justify structural feedback, and write review comments as observation, risk, fix, and justification. This makes critique objective and actionable.

Exercises

Pick one of your schemas (clinic or library) and conduct a self-review. Answer in full sentences.

1. Identify two domain terms that could be ambiguous and define them precisely.
2. Identify one key decision you would challenge and propose an alternative.
3. Identify one missing business constraint and write it as a constraint statement.
4. Identify one redundancy risk and state the dependency that reveals it.
5. Write three review comments using the observation/risk/fix/justification format.

Chapter 17

Requirements and Scope

Learning objectives

In this chapter you will learn how to define scope for a conceptual design. You will be able to separate requirements that shape meaning from those that belong to later implementation. You will practice writing clear scope boundaries and assumptions.

17.1 Why scope matters

A model is only as clear as its boundaries. If the scope is too wide, the model becomes vague. If it is too narrow, it becomes brittle. Scope is not a technical detail. It is a design decision about meaning.

17.2 What belongs in conceptual design

Conceptual design captures facts, rules, and relationships. It answers: what exists, what it means, and what is allowed. It does not answer how data will be stored or optimized.

17.2.1 Entities, attributes, and relationships

Entities represent identity. Attributes describe those entities. Relationships express associations and constraints. This is the core vocabulary of the model.

17.2.2 Rules and constraints

Rules belong in the conceptual model when they define valid states. A model without constraints is a story, not a design.

Example

In-scope statements A member is uniquely identified by `member_id`.
A loan must reference exactly one member and one copy.
A copy may have at most one active loan at a time.

17.3 What does not belong yet

Some topics are important, but not conceptual. They are deliberately out of scope in this phase.

17.3.1 Performance and storage

Indexes, caching, and partitioning are implementation concerns. They depend on workload and technology choices. They do not change meaning.

17.3.2 SQL and physical schemas

SQL expresses the model, but it is not the model. The conceptual design should be complete without a specific dialect.

17.3.3 User interfaces and workflows

UI workflows influence data needs, but they are not rules of the domain. Keep the model neutral and stable.

Example

Out-of-scope statements “Add an index on member_id.”
“Use a JSON column for tags.”
“Show a drop-down for status.”

17.4 Sources of requirements

Requirements come from people, documents, and existing systems. No single source is enough. You need triangulation.

17.4.1 Stakeholder interviews

Listen for nouns, verbs, and constraints. Ask for exceptions and edge cases. Those define the shape of the model.

17.4.2 Policies and artifacts

Policies describe rules. Forms and reports reveal attribute expectations. Existing databases reveal legacy assumptions.

17.5 Writing requirements as rules

A good rule is testable and specific. It avoids “usually” and “typically”. It uses explicit cardinality and optionality.

Principle

Principle Model meaning, not implementation.

17.6 Defining boundaries and assumptions

State what you are not modelling. State what you assume to be true. This makes the model stable and reviewable.

17.6.1 Boundary statements

Examples include regions, time ranges, and product lines. They reduce ambiguity and prevent scope creep.

17.6.2 Assumption log

Assumptions should be explicit and revisited. They are part of the model's contract.

Summary

Scope is a design decision, not a formality. Conceptual design includes entities, attributes, relationships, and rules. It excludes SQL, performance tuning, and UI decisions. Clear boundaries make the model coherent and reviewable.

Exercises

Define scope for a small domain of your choice.

1. Write five in-scope rules as precise statements.
2. List three out-of-scope concerns you will not model yet.
3. Identify two sources you would use to validate the rules.
4. Write three assumptions that might change later.
5. Explain how you would communicate scope to stakeholders.

Chapter 18

Modelling Nulls and Optional Data

Learning objectives

In this chapter you will learn the meaning of missing data. You will distinguish unknown values from not-applicable values. You will connect optionality in relationships to optionality in attributes.

18.1 Why nulls are a conceptual issue

Nulls are often treated as a storage detail. That is a mistake. A null encodes meaning. If the meaning is unclear, the data becomes ambiguous.

18.2 Optionality versus missingness

Optionality describes whether a relationship can be absent. Missingness describes whether an attribute value is unknown or not applicable. These are different ideas.

18.2.1 Optional relationships

An appointment must have a patient. A patient may have zero appointments. The relationship is optional from the patient side.

18.2.2 Optional attributes

A return date may not be present. This does not mean the loan is optional. It means the value is not yet known.

Example

Optional relationship vs optional attribute A patient may have zero appointments.
A loan may have a return date after it is returned.
These are different kinds of optionality.

18.3 Unknown versus not applicable

Unknown means the value exists but is not yet known. Not applicable means the value does not exist for that instance. This distinction should be explicit.

18.3.1 Unknown

“Unknown” is a temporary state. It should usually be resolved over time.

18.3.2 Not applicable

“Not applicable” is permanent for that instance. It should not be forced into a placeholder value.

Example

Unknown vs not applicable A patient’s insurance number may be unknown at registration.
A walk-in patient may not have an insurance number at all.

18.4 Temporal missingness

Some values are missing because the event has not occurred. A return date before return is unknown, not applicable. A graduation date before graduation is unknown. After graduation, it is known.

18.5 Defaults and placeholders

Defaults can hide missingness. Placeholders are often worse. They make meaning ambiguous. Use explicit rules instead.

Principle

Principle Every null must have a meaning.

18.6 Impact on constraints

Constraints must match the intended meaning. If a value can be unknown, allow it explicitly. If a value is not applicable, model the condition that makes it inapplicable.

18.6.1 Conceptual constraints

State whether the value is required, optional, or conditional. If conditional, write the rule in words.

18.6.2 Relational mapping

When you later map to relations, nullability follows these rules. Do not decide nullability based on convenience.

Summary

Nulls are not just storage markers. They encode meaning about unknown or not-applicable values. Optionality in relationships is not the same as missingness in attributes. Clear rules prevent silent ambiguity.

Exercises

Consider a domain such as a clinic or library.

1. List five attributes that might be missing.
2. For each, classify as unknown or not applicable.
3. Write a rule that explains when each value becomes known.
4. Identify one relationship that is optional and explain why.
5. Rewrite one ambiguous rule to remove null confusion.

Chapter 19

Decomposition Heuristics

Learning objectives

In this chapter you will learn how to split relations by meaning. You will recognize mixed-grain structures and repeating groups. You will practice decomposition without relying on convenience.

19.1 One relation, one meaning

A relation should describe one kind of fact. If it mixes meanings, it invites contradictions. This is the core heuristic.

Principle

Principle
Split by meaning, not by convenience.

19.2 Repeating groups and collections

If an attribute holds many values, the meaning is unclear. Lists hide multiplicity. They belong in a separate relation.

19.2.1 Multi-valued attributes

If an entity can have many phone numbers, the phone numbers are a collection. Collections deserve their own identity and constraints.

Example

Repeating group
Member(member_id, name, phone1, phone2, phone3)
Better meaning: **Member**(member_id, name) and **MemberPhone**(member_id, phone).

19.3 Mixed grain

A mixed-grain relation blends entity facts with event facts. This causes duplication and update anomalies.

19.3.1 Entity versus event

A person's name is stable. An appointment time is an event. Do not store them in the same relation unless the event is the entity.

Example

Mixed grain

$$\textbf{Appointment}(appointment\_id, start\_time, doctor\_name, patient\_name)$$

Better meaning:

$$\begin{aligned}&\textbf{Doctor}(doctor\_id, name),\\ &\textbf{Patient}(patient\_id, name),\\ &\textbf{Appointment}(appointment\_id, start\_time,\\ &\qquad doctor\_id \to \textbf{Doctor}, patient\_id \to \textbf{Patient})\end{aligned}$$

19.4 Derived data

Derived attributes can be useful, but they are not primary facts. If you include them, be explicit that they are derived. Otherwise they will be mistaken for ground truth.

19.5 Stability and change

If part of a relation changes at a different rate, you likely have two meanings. Split slow-changing facts from fast-changing facts. This keeps history consistent.

19.5.1 Time-varying attributes

When a value changes over time, consider a separate relation to track history. This is a conceptual decision before any SQL.

19.6 Decomposition checklist

Ask three questions:

- Does each attribute describe the same kind of fact?
- Can the attributes change together?
- Would a missing subset still make sense?

If any answer is no, decomposition is likely needed.

Summary

Decomposition is about meaning, not performance. Repeating groups, mixed grain, and time-varying facts signal a need to split. A single relation should represent a single kind of fact.

Exercises

Choose a small domain and draft a single relation that mixes facts.

1. Identify at least two different meanings in the relation.
2. Decompose it into two or more relations with clear meaning.
3. Describe the relationships between the new relations.
4. State one constraint that is easier to express after decomposition.
5. Explain how the decomposition improves clarity.

Chapter 20

Constraints Catalog

Learning objectives

In this chapter you will learn a structured catalog of constraints. You will distinguish uniqueness, inclusion, and domain constraints. You will practice expressing constraints without SQL.

20.1 Why a catalog helps

Constraints define valid states. A catalog ensures you do not miss common rule types. It also improves review discussions.

20.2 Uniqueness constraints

Uniqueness defines identity and prevents duplicates. It can apply to a single attribute or a combination.

20.2.1 Single-attribute uniqueness

Example: a member is uniquely identified by `member_id`. This is a primary identity rule.

20.2.2 Composite uniqueness

Composite uniqueness captures meaning like “no double booking.” It is a relationship-level rule expressed as a unique combination.

Example

Composite uniqueness
Rule: A doctor cannot have two appointments at the same start time.
Meaning: (doctor_id, start_time) is unique in **Appointment**.

20.3 Inclusion dependencies

Inclusion dependencies express referential integrity. They say one set of values must be contained in another.

20.3.1 Mandatory references

If a loan references a member, the member must exist. This is a core relational meaning, not a storage detail.

Example

Inclusion dependency

$$\textbf{Loan}.member\_id \rightarrow \textbf{Member}.member\_id$$

The referenced member must exist.

20.4 Domain constraints

Domain constraints restrict allowed values. They protect meaning at the attribute level.

20.4.1 Enumerations

Status values should come from a defined set. This prevents silent drift in meaning.

20.4.2 Value ranges

Dates, counts, and measures often have valid ranges. These are cheap but powerful constraints.

Principle

Principle
If a value has meaning, it has constraints.

20.5 Cross-row constraints

Some rules depend on multiple rows. They are often business constraints. They must still be stated conceptually.

20.5.1 No overlap

Examples include no overlapping reservations or appointments. State the rule in words even if enforcement is complex.

20.6 Temporal constraints

Time introduces ordering and validity windows. Rules like “return date after checkout date” are temporal. They should be explicit in the model.

Summary

Constraints are not optional. Use a catalog to ensure you cover uniqueness, inclusion, and domain rules. Cross-row and temporal constraints preserve real-world meaning.

Exercises

Build a constraint list for a small domain.

1. Write two uniqueness constraints (one composite).
2. Write two inclusion dependencies.
3. Write two domain constraints (enum or range).
4. Write one cross-row constraint.
5. Explain how each constraint protects meaning.

Chapter 21

Conclusion and Next Steps

Learning objectives

In this chapter you will consolidate the conceptual design workflow. You will see what is required to move toward implementation later. You will identify the artifacts needed for a clean handoff.

21.1 What you have built

You now have a conceptual model grounded in meaning. It includes entities, relationships, and constraints. It is independent of any SQL dialect.

21.2 From model to implementation

Implementation is a translation step, not a redesign. The model provides the rules. The implementation provides the mechanics.

21.2.1 Mapping as a disciplined translation

Entities become relations. Relationships become references or association relations. Constraints become enforceable checks.

21.3 Documentation as a deliverable

A model is only useful if it is shared and understood. Your documentation should be precise and minimal.

21.3.1 Core artifacts

At minimum, provide:

- A diagram for communication.
- A rule list with explicit constraints.
- A glossary of terms and relationships.

Example

Handoff package Diagram, rule list, constraint catalog, and a short glossary. These are enough for implementation without ambiguity.

21.4 Readiness checks

Before implementation, confirm these points. They prevent late surprises.

21.4.1 Model stability

Do key entities and rules remain consistent under edge cases? If not, return to requirements.

21.4.2 Stakeholder agreement

A model is valid only when stakeholders agree with the rules. Review is part of design, not an afterthought.

Principle

Principle Clarity comes before optimization.

21.5 What comes next (but not yet)

Implementation will introduce SQL, indexes, and performance choices. Those topics are important, but they belong to a later phase. For now, focus on meaning and correctness.

Example

Tease the next phase You will later choose keys, constraints, and indexes in SQL. Those choices should mirror the conceptual rules, not replace them.

21.6 Closing loop

A good model is a living artifact. It should be revisited as the domain evolves. Treat it as part of the system, not a one-off document.

Summary

You have a complete conceptual framework for relational design. The next step is implementation, but only after clarity and agreement. Documentation, rules, and constraints form a reliable handoff.

Exercises

Reflect on a model you have built or reviewed.

1. List three rules that were most critical to meaning.
2. Identify one place where the model was still ambiguous.
3. Write a short handoff checklist for a future implementation team.
4. Describe one risk if implementation starts too early.

Appendix A

SQL Appendix: Chapter 1 (Workflow)

A.1 DDL

```
CREATE TABLE member (
  member_id INT PRIMARY KEY AUTO_INCREMENT,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(200) NULL UNIQUE,
  joined_at DATE NOT NULL
) ENGINE=InnoDB;

CREATE TABLE loan (
  loan_id INT PRIMARY KEY AUTO_INCREMENT,
  member_id INT NOT NULL,
  checkout_date DATE NOT NULL,
  due_date DATE NOT NULL,
  return_date DATE NULL,
  CONSTRAINT fk_loan_member
    FOREIGN KEY (member_id) REFERENCES member(member_id)
) ENGINE=InnoDB;
```

A.2 Sample data

```
INSERT INTO member (full_name, email, joined_at)
VALUES
  ('Ana Costa', 'ana@example.com', '2026-01-10'),
  ('Rui Silva', NULL, '2026-01-12');

INSERT INTO loan (member_id, checkout_date, due_date, return_date)
VALUES
  (1, '2026-01-15', '2026-01-29', NULL),
  (1, '2025-12-01', '2025-12-15', '2025-12-14');
```

A.3 Example queries

```
-- Q1: Active loans per member
SELECT m.full_name, l.loan_id, l.checkout_date, l.due_date
FROM member m
```

```
JOIN loan l ON l.member_id = m.member_id
WHERE l.return_date IS NULL;

-- Q2: Overdue loans today
SELECT loan_id, member_id, due_date
FROM loan
WHERE return_date IS NULL AND due_date < CURRENT_DATE();
```

Appendix B

SQL Appendix: Chapter 2 (Core Concepts)

B.1 DDL

```
CREATE TABLE title (
  title_id INT PRIMARY KEY AUTO_INCREMENT,
  isbn VARCHAR(20) NULL UNIQUE,
  title_name VARCHAR(200) NOT NULL,
  author_name VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE copy (
  copy_id INT PRIMARY KEY AUTO_INCREMENT,
  barcode VARCHAR(50) NULL UNIQUE,
  status VARCHAR(20) NOT NULL,
  title_id INT NOT NULL,
  CONSTRAINT fk_copy_title
    FOREIGN KEY (title_id) REFERENCES title(title_id)
) ENGINE=InnoDB;
```

B.2 Sample data

```
INSERT INTO title (isbn, title_name, author_name)
VALUES
  ('9780131103627', 'The C Programming Language', 'Kernighan, Ritchie'),
  (NULL, 'Local History Archive', 'City Council');

INSERT INTO copy (barcode, status, title_id)
VALUES
  ('BC-001', 'available', 1),
  ('BC-002', 'on_loan', 1),
  ('BC-003', 'available', 2);
```

B.3 Example queries

```
-- Q1: Copies for a title
SELECT c.copy_id, c.barcode, c.status
FROM copy c
```

```
WHERE c.title_id = 1;

-- Q2: Titles with no copies
SELECT t.title_id, t.title_name
FROM title t
LEFT JOIN copy c ON c.title_id = t.title_id
WHERE c.copy_id IS NULL;
```

Appendix C

SQL Appendix: Chapter 3 (Keys and Identity)

C.1 DDL

```
CREATE TABLE student (
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(200) NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE course (
  course_id INT PRIMARY KEY AUTO_INCREMENT,
  code VARCHAR(20) NOT NULL UNIQUE,
  title VARCHAR(200) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE enrollment (
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  term VARCHAR(10) NOT NULL,
  grade VARCHAR(2) NULL,
  PRIMARY KEY (student_id, course_id, term),
  CONSTRAINT fk_enroll_student FOREIGN KEY (student_id) REFERENCES
  ↪  student(student_id),
  CONSTRAINT fk_enroll_course FOREIGN KEY (course_id) REFERENCES
  ↪  course(course_id)
) ENGINE=InnoDB;
```

C.2 Sample data

```
INSERT INTO student (full_name, email)
VALUES
  ('Marta Lima', 'marta@uni.edu'),
  ('Rui Nunes', NULL);

INSERT INTO course (code, title)
VALUES
  ('DB101', 'Database Design'),
```

```
  ('CS120', 'Programming Fundamentals');

INSERT INTO enrollment (student_id, course_id, term, grade)
VALUES
  (1, 1, '2025-FALL', 'A'),
  (1, 1, '2026-SPR', NULL),
  (2, 2, '2026-SPR', NULL);
```

C.3 Example queries

```
-- Q1: Enrollments per student and term
SELECT student_id, course_id, term
FROM enrollment
WHERE student_id = 1;

-- Q2: Students with duplicate emails (should be none)
SELECT email, COUNT(*)
FROM student
WHERE email IS NOT NULL
GROUP BY email
HAVING COUNT(*) > 1;
```

Appendix D

SQL Appendix: Chapter 4 (Conceptual to Relational)

D.1 DDL

```
CREATE TABLE customer (
  customer_id INT PRIMARY KEY AUTO_INCREMENT,
  full_name VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE product (
  product_id INT PRIMARY KEY AUTO_INCREMENT,
  sku VARCHAR(40) NOT NULL UNIQUE,
  name VARCHAR(200) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE `order` (
  order_id INT PRIMARY KEY AUTO_INCREMENT,
  placed_at DATETIME NOT NULL,
  customer_id INT NOT NULL,
  CONSTRAINT fk_order_customer FOREIGN KEY (customer_id) REFERENCES
  ↪  customer(customer_id)
) ENGINE=InnoDB;

CREATE TABLE order_line (
  order_id INT NOT NULL,
  product_id INT NOT NULL,
  quantity INT NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (order_id, product_id),
  CONSTRAINT fk_line_order FOREIGN KEY (order_id) REFERENCES `order`(order_id),
  CONSTRAINT fk_line_product FOREIGN KEY (product_id) REFERENCES
  ↪  product(product_id)
) ENGINE=InnoDB;
```

D.2 Sample data

```
INSERT INTO customer (full_name) VALUES ('Ana Costa');
INSERT INTO product (sku, name) VALUES ('BK-001', 'Design Book');
```

```
INSERT INTO `order` (placed_at, customer_id) VALUES ('2026-02-01 10:00:00', 1);
INSERT INTO order_line (order_id, product_id, quantity, unit_price)
VALUES (1, 1, 2, 35.00);
```

D.3 Example queries

```
-- Q1: Order summary with line totals
SELECT o.order_id, p.name, l.quantity, l.unit_price,
       l.quantity * l.unit_price AS line_total
FROM `order` o
JOIN order_line l ON l.order_id = o.order_id
JOIN product p ON p.product_id = l.product_id;

-- Q2: Orders for a customer
SELECT order_id, placed_at
FROM `order`
WHERE customer_id = 1;
```

Appendix E

SQL Appendix: Chapter 5 (Normalization)

E.1 DDL (normalized)

```
CREATE TABLE doctor (
  doctor_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL,
  specialty VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE patient (
  patient_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL,
  birth_date DATE NOT NULL
) ENGINE=InnoDB;

CREATE TABLE appointment (
  appointment_id INT PRIMARY KEY AUTO_INCREMENT,
  start_time DATETIME NOT NULL,
  doctor_id INT NOT NULL,
  patient_id INT NOT NULL,
  CONSTRAINT fk_appt_doctor FOREIGN KEY (doctor_id) REFERENCES
  ↪  doctor(doctor_id),
  CONSTRAINT fk_appt_patient FOREIGN KEY (patient_id) REFERENCES
  ↪  patient(patient_id)
) ENGINE=InnoDB;
```

E.2 Sample data

```
INSERT INTO doctor (name, specialty) VALUES
  ('Joana Reis', 'cardiology'),
  ('Hugo Pereira', 'pediatrics');

INSERT INTO patient (name, birth_date) VALUES
  ('Ana Costa', '1990-04-11'),
  ('Rui Silva', '1985-09-03');

INSERT INTO appointment (start_time, doctor_id, patient_id) VALUES
```

```
  ('2026-02-03 09:00:00', 1, 1),
  ('2026-02-03 09:30:00', 1, 2);
```

E.3 Example queries

```
-- Q1: Appointments with doctor and patient names
SELECT a.appointment_id, a.start_time, d.name AS doctor, p.name AS patient
FROM appointment a
JOIN doctor d ON d.doctor_id = a.doctor_id
JOIN patient p ON p.patient_id = a.patient_id;

-- Q2: Detect redundant specialty storage (should be in doctor only)
SELECT a.appointment_id, d.specialty
FROM appointment a
JOIN doctor d ON d.doctor_id = a.doctor_id;
```

Appendix F

SQL Appendix: Chapter 6 (Integrity Constraints)

F.1 DDL

```
CREATE TABLE room (
  room_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE doctor (
  doctor_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE appointment (
  appointment_id INT PRIMARY KEY AUTO_INCREMENT,
  start_time DATETIME NOT NULL,
  duration_min INT NOT NULL,
  status VARCHAR(20) NOT NULL,
  doctor_id INT NOT NULL,
  room_id INT NOT NULL,
  CONSTRAINT chk_duration CHECK (duration_min > 0),
  CONSTRAINT chk_status CHECK (status IN ('scheduled','cancelled')),
  CONSTRAINT fk_appt_doctor FOREIGN KEY (doctor_id) REFERENCES
  ↪  doctor(doctor_id),
  CONSTRAINT fk_appt_room FOREIGN KEY (room_id) REFERENCES room(room_id)
) ENGINE=InnoDB;
```

F.2 Sample data

```
INSERT INTO room (name) VALUES ('R1'), ('R2');
INSERT INTO doctor (name) VALUES ('Joana Reis'), ('Hugo Pereira');
INSERT INTO appointment (start_time, duration_min, status, doctor_id, room_id)
VALUES
  ('2026-02-05 09:00:00', 60, 'scheduled', 1, 1),
  ('2026-02-05 10:30:00', 30, 'cancelled', 1, 1);
```

F.3 Example queries

```
-- Q1: Find invalid durations (should be none)
SELECT appointment_id
FROM appointment
WHERE duration_min <= 0;

-- Q2: Detect overlapping appointments per room (conceptual validation)
SELECT a.appointment_id, b.appointment_id
FROM appointment a
JOIN appointment b
  ON a.room_id = b.room_id
 AND a.appointment_id < b.appointment_id
 AND a.status = 'scheduled'
 AND b.status = 'scheduled'
 AND a.start_time < DATE_ADD(b.start_time, INTERVAL b.duration_min MINUTE)
 AND b.start_time < DATE_ADD(a.start_time, INTERVAL a.duration_min MINUTE);
```

Appendix G

SQL Appendix: Chapter 7 (Validation)

G.1 DDL

```
CREATE TABLE patient (
  patient_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE doctor (
  doctor_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE appointment (
  appointment_id INT PRIMARY KEY AUTO_INCREMENT,
  start_time DATETIME NOT NULL,
  duration_min INT NOT NULL,
  status VARCHAR(20) NOT NULL,
  patient_id INT NOT NULL,
  doctor_id INT NOT NULL,
  CONSTRAINT fk_appt_patient FOREIGN KEY (patient_id) REFERENCES
  ↪  patient(patient_id),
  CONSTRAINT fk_appt_doctor FOREIGN KEY (doctor_id) REFERENCES
  ↪  doctor(doctor_id)
) ENGINE=InnoDB;
```

G.2 Sample data

```
INSERT INTO patient (name) VALUES ('Ana Costa'), ('Rui Silva');
INSERT INTO doctor (name) VALUES ('Joana Reis');
INSERT INTO appointment (start_time, duration_min, status, patient_id,
↪  doctor_id)
VALUES
  ('2026-02-06 09:00:00', 60, 'scheduled', 1, 1),
  ('2026-02-06 09:30:00', 30, 'scheduled', 2, 1);
```

G.3 Validation queries

```
-- Q1: Appointments per doctor
SELECT doctor_id, COUNT(*) AS appt_count
FROM appointment
GROUP BY doctor_id;

-- Q2: Overlap detection (should return empty for valid data)
SELECT a.appointment_id, b.appointment_id
FROM appointment a
JOIN appointment b
  ON a.doctor_id = b.doctor_id
 AND a.appointment_id < b.appointment_id
 AND a.status = 'scheduled'
 AND b.status = 'scheduled'
 AND a.start_time < DATE_ADD(b.start_time, INTERVAL b.duration_min MINUTE)
 AND b.start_time < DATE_ADD(a.start_time, INTERVAL a.duration_min MINUTE);
```

Appendix H

SQL Appendix: Chapter 8 (Naming and Documentation)

H.1 DDL with comments

```
CREATE TABLE member (
  member_id INT PRIMARY KEY AUTO_INCREMENT,
  full_name VARCHAR(120) NOT NULL,
  email VARCHAR(200) NULL UNIQUE,
  created_at DATETIME NOT NULL
) ENGINE=InnoDB
COMMENT='Member records with stable identifiers';

CREATE TABLE loan (
  loan_id INT PRIMARY KEY AUTO_INCREMENT,
  member_id INT NOT NULL,
  checkout_date DATE NOT NULL,
  due_date DATE NOT NULL,
  return_date DATE NULL,
  CONSTRAINT fk_loan_member
    FOREIGN KEY (member_id) REFERENCES member(member_id)
) ENGINE=InnoDB
COMMENT='Loan events; one row per borrowing';
```

H.2 Sample data

```
INSERT INTO member (full_name, email, created_at)
VALUES ('Ana Costa', 'ana@example.com', '2026-01-10 09:00:00');

INSERT INTO loan (member_id, checkout_date, due_date, return_date)
VALUES (1, '2026-01-20', '2026-02-03', NULL);
```

H.3 Example queries

```
-- Q1: Member loans with semantic date fields
SELECT m.full_name, l.checkout_date, l.due_date
FROM member m
JOIN loan l ON l.member_id = m.member_id;
```

Appendix I

SQL Appendix: Chapter 9 (Clinic Case Study)

I.1 DDL

```
CREATE TABLE patient (
  patient_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE doctor (
  doctor_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE room (
  room_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE appointment (
  appointment_id INT PRIMARY KEY AUTO_INCREMENT,
  start_time DATETIME NOT NULL,
  duration_min INT NOT NULL,
  status VARCHAR(20) NOT NULL,
  patient_id INT NOT NULL,
  doctor_id INT NOT NULL,
  room_id INT NULL,
  CONSTRAINT fk_appt_patient FOREIGN KEY (patient_id) REFERENCES
  ↪  patient(patient_id),
  CONSTRAINT fk_appt_doctor FOREIGN KEY (doctor_id) REFERENCES
  ↪  doctor(doctor_id),
  CONSTRAINT fk_appt_room FOREIGN KEY (room_id) REFERENCES room(room_id),
  CONSTRAINT chk_status CHECK (status IN ('scheduled','cancelled')),
  CONSTRAINT chk_duration CHECK (duration_min > 0)
) ENGINE=InnoDB;
```

I.2 Sample data

```
INSERT INTO patient (name) VALUES ('Ana Costa'), ('Rui Silva');
INSERT INTO doctor (name) VALUES ('Joana Reis'), ('Hugo Pereira');
INSERT INTO room (name) VALUES ('R1'), ('R2');

INSERT INTO appointment (start_time, duration_min, status, patient_id,
↪  doctor_id, room_id)
VALUES
  ('2026-02-06 09:00:00', 60, 'scheduled', 1, 1, 1),
  ('2026-02-06 10:30:00', 30, 'cancelled', 2, 1, NULL);
```

I.3 Example queries

```
-- Q1: Doctor schedule
SELECT a.start_time, a.duration_min, a.status, r.name AS room
FROM appointment a
LEFT JOIN room r ON r.room_id = a.room_id
WHERE a.doctor_id = 1
ORDER BY a.start_time;

-- Q2: Room overlap check
SELECT a.appointment_id, b.appointment_id
FROM appointment a
JOIN appointment b
  ON a.room_id = b.room_id
 AND a.appointment_id < b.appointment_id
 AND a.status = 'scheduled'
 AND b.status = 'scheduled'
 AND a.start_time < DATE_ADD(b.start_time, INTERVAL b.duration_min MINUTE)
 AND b.start_time < DATE_ADD(a.start_time, INTERVAL a.duration_min MINUTE);
```

Appendix J

SQL Appendix: Chapter 10 (Library Case Study)

J.1 DDL

```
CREATE TABLE member (
  member_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE branch (
  branch_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE title (
  title_id INT PRIMARY KEY AUTO_INCREMENT,
  isbn VARCHAR(20) NULL UNIQUE,
  title_name VARCHAR(200) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE copy (
  copy_id INT PRIMARY KEY AUTO_INCREMENT,
  status VARCHAR(20) NOT NULL,
  title_id INT NOT NULL,
  branch_id INT NOT NULL,
  CONSTRAINT fk_copy_title FOREIGN KEY (title_id) REFERENCES title(title_id),
  CONSTRAINT fk_copy_branch FOREIGN KEY (branch_id) REFERENCES
  ↪ branch(branch_id)
) ENGINE=InnoDB;

CREATE TABLE loan (
  loan_id INT PRIMARY KEY AUTO_INCREMENT,
  checkout_date DATE NOT NULL,
  due_date DATE NOT NULL,
  return_date DATE NULL,
  member_id INT NOT NULL,
  copy_id INT NOT NULL,
  CONSTRAINT fk_loan_member FOREIGN KEY (member_id) REFERENCES
  ↪ member(member_id),
```

```
  CONSTRAINT fk_loan_copy FOREIGN KEY (copy_id) REFERENCES copy(copy_id)
) ENGINE=InnoDB;

CREATE TABLE reservation (
  reservation_id INT PRIMARY KEY AUTO_INCREMENT,
  request_date DATE NOT NULL,
  status VARCHAR(20) NOT NULL,
  member_id INT NOT NULL,
  title_id INT NOT NULL,
  CONSTRAINT fk_res_member FOREIGN KEY (member_id) REFERENCES
  ↪  member(member_id),
  CONSTRAINT fk_res_title FOREIGN KEY (title_id) REFERENCES title(title_id)
) ENGINE=InnoDB;

CREATE TABLE fine (
  fine_id INT PRIMARY KEY AUTO_INCREMENT,
  amount DECIMAL(10,2) NOT NULL,
  assessed_date DATE NOT NULL,
  status VARCHAR(20) NOT NULL,
  member_id INT NOT NULL,
  loan_id INT NOT NULL,
  CONSTRAINT fk_fine_member FOREIGN KEY (member_id) REFERENCES
  ↪  member(member_id),
  CONSTRAINT fk_fine_loan FOREIGN KEY (loan_id) REFERENCES loan(loan_id)
) ENGINE=InnoDB;

CREATE TABLE payment (
  payment_id INT PRIMARY KEY AUTO_INCREMENT,
  paid_at DATETIME NOT NULL,
  amount DECIMAL(10,2) NOT NULL,
  fine_id INT NOT NULL,
  CONSTRAINT fk_payment_fine FOREIGN KEY (fine_id) REFERENCES fine(fine_id)
) ENGINE=InnoDB;
```

J.2 Sample data

```
INSERT INTO member (name) VALUES ('Ana Costa');
INSERT INTO branch (name) VALUES ('Central');
INSERT INTO title (isbn, title_name) VALUES ('9780131103627', 'The C
↪  Programming Language');
INSERT INTO copy (status, title_id, branch_id) VALUES ('available', 1, 1);
INSERT INTO loan (checkout_date, due_date, return_date, member_id, copy_id)
VALUES ('2026-02-01', '2026-02-15', NULL, 1, 1);
```

J.3 Example queries

```
-- Q1: Active loans per member
SELECT l.loan_id, l.copy_id
FROM loan l
WHERE l.member_id = 1 AND l.return_date IS NULL;

-- Q2: Reservation queue for a title
```

```
SELECT reservation_id, member_id, request_date
FROM reservation
WHERE title_id = 1
ORDER BY request_date ASC;
```

Appendix K

SQL Appendix: Chapter 11 (State and Time)

K.1 DDL

```
CREATE TABLE copy (
  copy_id INT PRIMARY KEY AUTO_INCREMENT,
  status VARCHAR(20) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE loan (
  loan_id INT PRIMARY KEY AUTO_INCREMENT,
  copy_id INT NOT NULL,
  checkout_date DATE NOT NULL,
  return_date DATE NULL,
  CONSTRAINT fk_loan_copy FOREIGN KEY (copy_id) REFERENCES copy(copy_id)
) ENGINE=InnoDB;
```

K.2 Sample data

```
INSERT INTO copy (status) VALUES ('available'), ('on_loan');
INSERT INTO loan (copy_id, checkout_date, return_date)
VALUES (2, '2026-02-01', NULL);
```

K.3 Example queries

```
-- Q1: Derive active state from events
SELECT c.copy_id,
       CASE WHEN EXISTS (
         SELECT 1 FROM loan l
         WHERE l.copy_id = c.copy_id AND l.return_date IS NULL
       ) THEN 'on_loan' ELSE 'available' END AS derived_status
FROM copy c;

-- Q2: Detect mismatch if status is stored
SELECT c.copy_id, c.status
FROM copy c
WHERE c.status = 'available'
```

```
AND EXISTS (SELECT 1 FROM loan l WHERE l.copy_id = c.copy_id AND
↪  l.return_date IS NULL);
```

Appendix L

SQL Appendix: Chapter 12 (Relationships Advanced)

L.1 DDL (subtypes)

```
CREATE TABLE payment (
  payment_id INT PRIMARY KEY AUTO_INCREMENT,
  amount DECIMAL(10,2) NOT NULL,
  created_at DATETIME NOT NULL
) ENGINE=InnoDB;

CREATE TABLE card_payment (
  payment_id INT PRIMARY KEY,
  last4 CHAR(4) NOT NULL,
  auth_code VARCHAR(40) NOT NULL,
  CONSTRAINT fk_card_payment FOREIGN KEY (payment_id) REFERENCES
  ↪  payment(payment_id)
) ENGINE=InnoDB;

CREATE TABLE bank_transfer (
  payment_id INT PRIMARY KEY,
  iban VARCHAR(34) NOT NULL,
  reference VARCHAR(60) NOT NULL,
  CONSTRAINT fk_bank_transfer FOREIGN KEY (payment_id) REFERENCES
  ↪  payment(payment_id)
) ENGINE=InnoDB;
```

L.2 Sample data

```
INSERT INTO payment (amount, created_at) VALUES (35.00, '2026-02-01 10:00:00');
INSERT INTO card_payment (payment_id, last4, auth_code) VALUES (1, '1234',
↪  'AUTH-001');
```

L.3 Example queries

```
-- Q1: Payments with subtype details
SELECT p.payment_id, p.amount, cp.last4, bt.iban
FROM payment p
LEFT JOIN card_payment cp ON cp.payment_id = p.payment_id
```

```
LEFT JOIN bank_transfer bt ON bt.payment_id = p.payment_id;
```

Appendix M

SQL Appendix: Chapter 13 (Functional Dependencies)

M.1 DDL (decomposed)

```
CREATE TABLE specialty (
  specialty_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE doctor (
  doctor_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL,
  specialty_id INT NOT NULL,
  CONSTRAINT fk_doctor_specialty FOREIGN KEY (specialty_id) REFERENCES
  ↪  specialty(specialty_id)
) ENGINE=InnoDB;
```

M.2 Sample data

```
INSERT INTO specialty (name) VALUES ('cardiology');
INSERT INTO doctor (name, specialty_id) VALUES ('Joana Reis', 1);
```

M.3 Example queries

```
-- Q1: Doctor with specialty name
SELECT d.doctor_id, d.name, s.name AS specialty
FROM doctor d
JOIN specialty s ON s.specialty_id = d.specialty_id;
```

Appendix N

SQL Appendix: Chapter 14 (Normalization Patterns)

N.1 DDL (patterns)

```
CREATE TABLE role (
  role_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(80) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE user_account (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  full_name VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE user_role (
  user_id INT NOT NULL,
  role_id INT NOT NULL,
  valid_from DATE NOT NULL,
  valid_to DATE NULL,
  PRIMARY KEY (user_id, role_id, valid_from),
  CONSTRAINT fk_user_role_user FOREIGN KEY (user_id) REFERENCES
  ↪  user_account(user_id),
  CONSTRAINT fk_user_role_role FOREIGN KEY (role_id) REFERENCES role(role_id)
) ENGINE=InnoDB;

CREATE TABLE setting_definition (
  setting_key VARCHAR(60) PRIMARY KEY,
  description VARCHAR(200) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE entity_setting (
  entity_type VARCHAR(40) NOT NULL,
  entity_id INT NOT NULL,
  setting_key VARCHAR(60) NOT NULL,
  setting_value VARCHAR(200) NOT NULL,
  PRIMARY KEY (entity_type, entity_id, setting_key),
  CONSTRAINT fk_setting_key FOREIGN KEY (setting_key) REFERENCES
  ↪  setting_definition(setting_key)
) ENGINE=InnoDB;
```

N.2 Sample data

```
INSERT INTO role (name) VALUES ('admin'), ('librarian');
INSERT INTO user_account (full_name) VALUES ('Ana Costa');
INSERT INTO user_role (user_id, role_id, valid_from)
VALUES (1, 2, '2026-01-01');

INSERT INTO setting_definition (setting_key, description)
VALUES ('max_loans', 'Maximum active loans per member');
INSERT INTO entity_setting (entity_type, entity_id, setting_key, setting_value)
VALUES ('member', 1, 'max_loans', '5');
```

N.3 Example queries

```
-- Q1: Active roles today
SELECT ur.user_id, r.name
FROM user_role ur
JOIN role r ON r.role_id = ur.role_id
WHERE ur.valid_to IS NULL OR ur.valid_to >= CURRENT_DATE();

-- Q2: Settings for one member
SELECT setting_key, setting_value
FROM entity_setting
WHERE entity_type = 'member' AND entity_id = 1;
```

Appendix O

SQL Appendix: Chapter 15 (Design Trade-offs)

O.1 DDL (denormalized read model)

```
CREATE TABLE loan (
  loan_id INT PRIMARY KEY AUTO_INCREMENT,
  member_id INT NOT NULL,
  checkout_date DATE NOT NULL,
  due_date DATE NOT NULL,
  return_date DATE NULL
) ENGINE=InnoDB;

-- Denormalized read model (trade-off example)
CREATE TABLE loan_read_model (
  loan_id INT PRIMARY KEY,
  member_id INT NOT NULL,
  due_date DATE NOT NULL,
  overdue_flag TINYINT(1) NOT NULL DEFAULT 0
) ENGINE=InnoDB;
```

O.2 Sample data

```
INSERT INTO loan (member_id, checkout_date, due_date, return_date)
VALUES (1, '2026-01-10', '2026-01-24', NULL);

INSERT INTO loan_read_model (loan_id, member_id, due_date, overdue_flag)
VALUES (1, 1, '2026-01-24', 0);
```

O.3 Example queries

```
-- Q1: Read model usage
SELECT loan_id, member_id, overdue_flag
FROM loan_read_model
WHERE overdue_flag = 1;

-- Q2: Recompute overdue flag (derived truth)
SELECT loan_id,
       CASE WHEN return_date IS NULL AND due_date < CURRENT_DATE()
```

```
            THEN 1 ELSE 0 END AS derived_overdue
FROM loan;
```

Appendix P

SQL Appendix: Chapter 16 (Design Review)

P.1 DDL (flawed schema)

```
CREATE TABLE appointment_record (
  appointment_id INT PRIMARY KEY AUTO_INCREMENT,
  start_time DATETIME NOT NULL,
  doctor_name VARCHAR(120) NOT NULL,
  doctor_specialty VARCHAR(120) NOT NULL,
  patient_name VARCHAR(120) NOT NULL
) ENGINE=InnoDB;
```

P.2 DDL (reviewed fix)

```
CREATE TABLE doctor (
  doctor_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL,
  specialty VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE patient (
  patient_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(120) NOT NULL
) ENGINE=InnoDB;

CREATE TABLE appointment (
  appointment_id INT PRIMARY KEY AUTO_INCREMENT,
  start_time DATETIME NOT NULL,
  doctor_id INT NOT NULL,
  patient_id INT NOT NULL,
  CONSTRAINT fk_appt_doctor FOREIGN KEY (doctor_id) REFERENCES
  ↪  doctor(doctor_id),
  CONSTRAINT fk_appt_patient FOREIGN KEY (patient_id) REFERENCES
  ↪  patient(patient_id)
) ENGINE=InnoDB;
```

P.3 Example queries

```
-- Q1: Detect redundancy in flawed schema (same doctor_name repeated)
SELECT doctor_name, COUNT(*)
FROM appointment_record
GROUP BY doctor_name
HAVING COUNT(*) > 1;

-- Q2: Correct join after fix
SELECT a.appointment_id, a.start_time, d.name, p.name
FROM appointment a
JOIN doctor d ON d.doctor_id = a.doctor_id
JOIN patient p ON p.patient_id = a.patient_id;
```

Figure Index

Bibliography

[1] C. J. Date. *An Introduction to Database Systems*. 8th ed., Addison-Wesley, 2004.

[2] R. Elmasri and S. B. Navathe. *Fundamentals of Database Systems*. 7th ed., Pearson, 2016.